ACI UNIVERSAL PAYMENTS℠

Introduction Manual

NET24-XPNET™

Release 4.0

February 2020

Notices

ACI Worldwide

Offices in principal cities throughout the world.

www.aciworldwide.com

Americas +1 402 390 7600

Asia Pacific +65 6334 4843

Europe, Middle East, Africa +44 (0) 1923 816393

Contents

About this publication..... 6
What's new in this publication..... 6

Section 1: Getting started..... 8
XPNET architecture..... 8
XPNET nodes..... 8
HP NonStop systems..... 8
XPNET system concepts..... 8
XPNET process..... 8
Stations..... 9
Lines..... 9
Processes..... 9
Links..... 9
Dests..... 9
Functions within the XPNET process..... 10
XPNET network control..... 11
System configuration..... 11
XPNET and the HP NonStop Event Management Service..... 11
XPNET licensing scheme..... 12

Section 2: Overview of an XPNET process..... 13
NonStop facilities..... 13
Application process management..... 14
Process startup..... 14
Startup messages..... 14
Processes run as high PIN..... 14
State machines..... 15
Memory management..... 16
Queue facilities..... 17
Queue management..... 17
Application process queue management..... 19
Message routing..... 19
Message routing destinations..... 20
Destination string routing..... 20
Using an Integrated Endpoint process..... 20
Data communications..... 21
Supported protocols..... 21
Event management facilities..... 22
XPNET EMS implementation features..... 22
Event message types from the XPNET process..... 22
Event messages suppression..... 23
Audit facilities..... 23
Message traffic audit selection..... 23
Audit files..... 23
Audit buffering..... 24
Audit file reading utility..... 25
Debugging facilities..... 25
Event messages..... 25

Audits 26
TRACE LINE command 26
TRACE NODE command 26
TRACE NODE SSL command 26
Debug option 26
Process information 27
#DIAG-DEST internal diagnostic destination 27
Application programming interfaces 27

Section 3: Network control 28
The network control environment 28
NCPI Server process 30
NCP Server process 30
NCS Server process 31
Operator interfaces 31
Network control facilities 31
X.25 Network Inquiry subsystem screen 33
Security and operator interfaces 33
Network control commands 33
Environment commands 33
Inquiry commands 34
Object management commands 34
Network control security 35
Command access limitations 35
Command auditing 36
Access limitations and auditing commands 36
Network Control Security Profile File 36
Network Control Security Specification File 36
Event log files 36
Event log files viewing 37
Event message content 37

Section 4: Utilities 39
Command Profile Configuration Generator (CPCGEN) utility 39
Extended Network Configuration Generator (XNCGEN) utility 39
License Verification utility (LIVERIFY) 40
Network Audit Read utilities (NETADRD and NETADRT) 40
Network Dump Read utility (NETDMPRD) 41
Network Configuration Journal Read utility (NETCJRD) 42
Release procedure macro (RELPROC) 42
Routing Profile Configuration Generator (RPCGEN) utility 43

Section 5: ACI service-based architecture support 44
Terminology 44
Overview 44
Well-known services 45
Service locators 45
Service consumers 46
Service providers 46
What's inside 46
Processing 46
Synchronous versus asynchronous processing 46
Implementation tasks 47

Setting up XPNET security....47
Define the stations in the station pool....49
Configure the TCP/IP connection between XPNET and the UPP dynamic service registry (DSR)....49
Start the TCP/IP connection between XPNET and the UPP DSR....50
Add a SVLOC process under XPNET....50

Appendix A: Third-party software license agreements....51
ANTLR Software Rights Notice....51
Apache License 1.1....52
Apache License 2.0....53
BSD 3-clause "New" or "Revised" License....55
Exoffice License....56
JTA 101 Sun License....56
Saxpath License....58
Sun Java Message Service 1.0.2 License....58
Tyrex License (Vovida License +)....61

About this publication

This publication provides an introduction to NET24-XPNET ™, a product of ACI Worldwide Inc. The XPNET process and functions within the XPNET process are described, as well as processes and subsystems with which the XPNET process interacts. This publication also contains a brief introduction to configuring an XPNET system, the XPNET implementation of HP NonStop Event Management Services (EMS), network control, and utilities that are part of the NET24-XPNET product. This publication is designed primarily for ACI customers who have purchased the NET24-XPNET product.

Audience

This publication provides an overview of the concepts and facilities of the NET24-XPNET product for executives and technical personnel who are involved with an XPNET system. Readers of this publication should have some familiarity with data processing operations; however, knowledge of HP NonStop computers and ACI software is not necessary.

Hardware-specific variations

NET24-XPNET publications contain some references to products and utilities that vary depending on which HP platform is in use. This section explains how references to products and utilities that vary by HP platform are documented.

References to Inspect imply platform-specific versions of the Inspect product, such as eInspect on the HP NonStop NS-series platform, which is required for debugging TNS/E processes.

The ACI Sudoterm virtual hometerm utility is discontinued for the HP NonStop NS-series platform. Customers should use the HP Virtual Hometerm Subsystem (VHS) product instead of the Sudoterm utility on the NS-series platform. The Sudoterm utility is still available for use on earlier platforms (for example, NonStop S-series).

On the HP NonStop NS-series platform, the HP Object Code Accelerator (OCA) utility is used to accelerate non-native objects. On earlier platforms (for example, NonStop S-series), the HP AXCEL utility is used to perform this function.

What's new in this publication

The following highlights the major changes that have been made in the most recent update to the *NET24-XPNET Introduction Manual*.

Release 4.0, February 2020

Section	Description
1	If a license is more than 180 days expired, the system will stop until a new license is obtained.
4	Added NETADRT to the "Network Audit Read utility" topic
Appendix	Removed third-party licenses for Expat XML parser and HP NonStop as they are no longer being distributed with XPNET. Added licenses for 11 third-party software packages.

Release 3.4, December 2017

These changes update the publication to Release 3.4.

Section	Major changes
Section 1	Removes reference to the *NET24-XPNET Management Programming Manual* and replaces it with reference to *NET24-XPNET Network Control Operations Guide* in the topic "XPNET network control."
Section 2	Adds information about internal audit process versus external audit process to the "Auditing facilities" subtopics.
Section 3	Removes reference to the *NET24-XPNET Management Programming Manual* and replaces it with reference to *NET24-XPNET Network Control Operations Guide* in the topic "Management programming interface."
Appendix A	Updates the Exolab software license.

Section 1: Getting started

The NET24-XPNET product is a standard part of the NET24-MQA (message queueing architecture) family of products. This manual introduces the concepts and features of the NET24-XPNET product to managers and technical personnel using an XPNET system.

XPNET architecture

The NET24-XPNET product has a modular design. Because of this design, XPNET systems can expand to support increasing message volumes. Two elements are part of a basic architecture common to all XPNET systems: the XPNET node and the HP NonStop system. They provide the structure, or framework, for the application software managed by the XPNET process.

XPNET nodes

The XPNET node is the fundamental building block of an XPNET system. It consists of an XPNET process and its associated stations, lines, application processes, services and dests. The XPNET process and its application processes work together to accomplish message processing.

XPNET nodes can be replicated and distributed as required to support processing and operational demands. Each XPNET node has the ability to communicate with other XPNET nodes. Because of these features, an XPNET system is highly scalable.

HP NonStop systems

The HP NonStop system refers to the physical HP computer. The HP NonStop system executes the XPNET process and application software. HP NonStop systems communicate with one another using the HP NonStop Expand product, thereby functioning as a distributed processing system.

The HP NonStop network can consist of any number of HP NonStop systems and can include any available processor types or mix of processor types. The XPNET system can span multiple HP NonStop systems without affecting or altering the operational interfaces and without impacting connectivity between processes, stations, and lines in the configuration.

XPNET system concepts

With the NET24-XPNET product, network functions are segregated from application functions to achieve a separation of responsibilities along functional lines. The NET24-XPNET product handles network-level functions, such as communicating message or transaction data from one computer network to another. Application programs are responsible for application-specific functions, such as authorizing transactions or performing specific message processing.

Each system differs based on the applications involved, the operational demands of the system participants, and geographic requirements. However, common system components and a common system structure, or architecture, are inherent in any system. These components consist of stations, communications lines, processes, links, services and dests controlled by one or more XPNET processes.

XPNET process

The XPNET process is the controlling process for the XPNET node, managing a user-configured group of stations, lines, processes, services, dests, and links to other XPNET nodes. The XPNET process also manages the message flow between entities in the XPNET system. Each XPNET node includes one XPNET process.

The XPNET process is responsible for the following functions:

- Driving the data communications lines
- Controlling the stations, lines, and processes defined to it
- Routing messages
- Managing message queues
- Communicating with other XPNET processes
- Enabling continuous processing
- Running system audits
- Monitoring the network (when problems occur with stations, lines, or processes, the XPNET process reports the situation)
- Generating statistics
- Interfacing to the operating system
- Generating and writing event messages to the HP NonStop Event Management Service (EMS)

Stations

A station is a physical point in the XPNET system that sends or receives messages. Stations are abstractions of external, independently addressable endpoints. Remote devices and connections to other platforms are stations.

Stations can be added dynamically in the internal TCP/IP protocol using a template. When the station startup command is given the command checks to see if it is a template station. If it is, the specified number of stations are started with system-generated names. The stations remain in Started or Starting state until a STOP command is issued.

Lines

The XPNET process sends messages to stations throughout the system using communications lines. Messages transmitted over communications lines are governed by line protocols used within the industry. Protocols ensure an orderly exchange of data from one station to another by enforcing a standard format and procedures for transmitting data.

Processes

A software process executes a coded set of instructions contained in a software program. It is a running version of a software program that performs the functions specified in the program. Processes access a variety of data files for information necessary to execute the software programs. The XPNET process performs network-level functions and application processes perform application-level functions.

You can configure single or multiple copies of each application process. Multiple copies of an application process enable you to maintain efficient performance when processing demands exceed the capacity of a single application process.

Links

Links identify the other XPNET nodes with which an XPNET node can communicate. The basic function of the link is to provide a connecting path to the XPNET process for another XPNET node. Each XPNET process is responsible for establishing and maintaining its link with the other XPNET processes.

Dests

Dests represent entries in the destination table to which messages can be routed within the network. Dests can be local processes and stations, remote processes and stations, and services. The dest object provides a way to inquire on and configure routing table entries directly.

Dests can be unknown (typically due to a configuration error or system outage). By default all dests are dynamic and added as needed. Dest objects can be added to the NEF which will be used as a template when the dest is added to memory. An example might be wanting to configure specific QAT, QMI, QMT values for a service.

Functions within the XPNET process

The XPNET process comprises a number of bound modules. Each module is responsible for a specific function. Following are the bound modules that constitute the XPNET process:

Audit module	Handles network audits. Using information from the Network Environment File (NEF), the Executive module determines whether a message must be audited at specific points during message processing. If auditing is required, the Audit module audits the message.
Data Communications (Datacom) module	Handles data communications within the XPNET system. Support for specific protocols is bound into the Data Communications module. When a message is inbound or outbound on a line, the XPNET process uses its internal tables to determine the protocol being used on that line. The XPNET process then passes the message to the appropriate protocol-specific module, which processes the message. For a list of supported protocols, see *Supported protocols*.
Executive module	Handles initialization and message flow. The Executive module determines which XPNET process module should perform a task and passes control of the task to the appropriate module. When the task is completed, processing control returns to the Executive module. The Executive module then dispatches the next task.
Gateway module	Handles communication between XPNET processes. The Gateway module provides the link between XPNET nodes. It also handles dynamic (normal) and service based routing logic.
Interprocess Communications module	Creates and controls application processes. The Interprocess Communications module also handles the message traffic that is destined for or arrives from application processes.
Logging module	Writes XPNET process event messages to the HP NonStop Event Management Service (EMS).
Memory Manager module	Handles memory allocation and de-allocation for the XPNET process.
Network Control Facility module	Defines the network control interface for the XPNET system. The Network Control Facility provides a command set and interface used for network control of system objects.
NonStop module	Handles checkpoints to the backup XPNET process, if one is running. If the XPNET process is running in hot backup mode, the NonStop module checkpoints global space and stack space to its backup at the completion of every I/O, in addition to file control block information or local data space segments that have changed since the last checkpoint. If the XPNET process is running in warm backup mode, the NonStop module makes a single checkpoint to the backup XPNET process at initialization.
Queuing module	Handles the queuing of messages for objects in the network. When messages arrive at the XPNET process faster than an object can accept them, the Queuing module queues each message until the object can accept it. This module also performs the network overload logic.

XPNET network control

All network control functions in an XPNET system are handled by a common interface and implemented in a consistent fashion. The network command structure is built around common conventions and keywords.

The network control subsystem provides a common command and control interface to an XPNET system. Standard interfaces are through a formatted screen or an interactive prompt. These interfaces include the Network Control Supervisor (NCS) screen and the Network Control Point Communications (NCPCOM) utility. Network control commands are used to define, manage, and change objects. Inquiry commands that enable you to see the current settings for an object, and status or statistics commands that enable you to see the current status or accumulated statistics for an object are also available.

Because the network control interface is documented and supported, customers can construct alternate operator or programmatic interfaces to meet the needs of their business environment.

XPNET network control is described in more detail in *Network control*. The *NET24-XPNET Network Control Operations Guide* fully documents the network control facilities and the *NET24-XPNET Network Control Command Reference Manual* documents the commands that can be issued from them.

System configuration

The XPNET process and the application processes, stations, links, services, dests, and lines defined to it form an XPNET node. An XPNET node is the most basic part of an XPNET system.

An XPNET node and all of its component parts can be configured in real time. This includes lines, stations, application processes, services, dests, and links to other XPNET nodes. Once you have configured and started your first XPNET node, there is no need to bring the system down to make configuration changes or to add components to the system. Configuration commands are entered from the network control interface.

XPNET and the HP NonStop Event Management Service

The HP NonStop Event Management Service (EMS) is a component of the HP NonStop Distributed Systems Management (DSM) family of products. These products assist in the management of operating systems and application programs (such as an XPNET system) running on HP NonStop computers. EMS provides a consistent, flexible method to capture event messages generated by the HP NonStop system, XPNET, and application subsystems.

The primary purpose of EMS is to collect and distribute event messages. Event messages are unsolicited messages generated by the HP NonStop system, XPNET, and application subsystems. EMS primary and alternate collectors accept event messages and write them to event log files. EMS distributor processes provide for the selection and distribution of event messages.

Tokens define the information contained in an event message. A *token* is a position-independent data element that represents a piece of information in the event message. Each token combines with other tokens to create an event message. Tokens provide information used in the message itself (for example, the event message number or process ID) as well as information used to properly perform a function (for example, the Emphasis token specifies whether critical messages are highlighted in the display).

Event messages generated by the XPNET process are compatible with EMS because the XPNET process sends event messages to EMS in tokenized form. Application processes managed by the XPNET process can also generate event messages. The application process can generate an event message compatible with EMS or call a library procedure to make the event message compatible with EMS. The need to call a library procedure depends on the application programming interface (API) in which the application process was written.

For information on the content of event log files, see *Event log files*. For more information on the XPNET implementation of EMS, see the *NET24-XPNET Event Management Manual*.

XPNET licensing scheme

The XPNET licensing scheme uses a digital signature-based license issued to each customer. The XPNET process reads the license to determine which systems it can run on, which protocol modules it can use, and when it expires. The licensing scheme has the following features.

- Customers are provided a licensing certificate to place on the subvolume that holds the XPNET object. The certificate unlocks specific protocol modules, indicates which HP NonStop nodes XPNET can run on, and has an expiration date. The customer can view the certificate to query or verify its contents.
- The XPNET licensing scheme provides the ability to deliver all internal XPNET protocol modules to all customers. The customer has the option to link in as many protocols as they can anticipate using, whether the protocols are currently licensed or not. New protocols can be licensed, or the use of existing products can be extended, without bringing down the XPNET node.
- New license certificates can be e-mailed to the customer and applied while the network is up and running. An ONCF command is provided to check and install the new certificate. If there is a problem with a new certificate, XPNET does not fail, but continues to use the previous certificate.
- XPNET has been designed to log events for all foreseen licensing-related problems. The customer is notified well in advance when a license approaches expiration. If the license expires, an event is logged at regular intervals to indicate that a new license must be ordered. If a new license is not in place within 180 days after the expiration date, the XPNET node will log event 6480 and stop. Once this happens, a new valid license file will need to be installed to be able to restart the node(s).

Section 2: Overview of an XPNET process

The XPNET process is an executable program that runs under the control of the HP NonStop operating system. A running XPNET process accepts and processes the operating system process startup message.

The XPNET process handles its functions asynchronously using nowait I/O. Multiple reads and writes can be outstanding concurrently to data communications lines and processes. The XPNET process handles completions to these I/O operations as they occur.

To support service-based architecture (SBA) and business service integration (BSI), XPNET supports synchronous I/O. With this functionality, a message can be sent through XPNET to a process, station (UPP integration only), or service. This simplifies the configuration, the application only needs a symbolic name. If this is a service, there could be 10, 100, or 1000+ providers and the application does not need to be aware of this. XPNET utilizes its normal service based routing logic. This accomplished with a bit in the call to NETINIT or NETLIB_INIT and a procedure call (netlib_sync_send_recv in XNLIB or netsyncwritex in SKELB).

NonStop facilities

The XPNET process supports the HP NonStop architecture by creating a backup XPNET process and issuing checkpoints to the backup. The XPNET process supports different levels of processing, as described in the following table.

Backup option	Description
Hot	Application processes are opened by both the primary and backup XPNET processes. The primary XPNET process checkpoints all messages sent to or received from any destination and all configuration changes to the backup XPNET process. If the primary XPNET process fails, the backup XPNET process resends all messages in progress at the last checkpoint and carries them forward to completion.
Warm	The primary XPNET process makes a single checkpoint to the backup XPNET process at initialization. If the primary XPNET process fails, the backup XPNET process restarts all stations, processes (ADOPT OFF), and links that were running at the time the primary XPNET process failed. The backup XPNET process reads the Network Environment File (NEF) for configuration changes made since the last initialization. However, because messages are not checkpointed, messages in progress at the time of failure are lost. If the process is associative (ADOPT ON), the node re-established connectivity with the process (does not stop and restart it).
No backup	The primary XPNET process does not start a backup XPNET process. If the primary XPNET process fails, there is no backup XPNET process to take over processing.

You specify the backup option at the XPNET node level. In a system with multiple XPNET nodes, you can configure all nodes with the same kind of backup or configure a different backup option for each node, based on business or application requirements.

The XPNET process uses the operating system procedures to create and maintain a current backup. The XPNET process creates a backup at node startup if you have specified different CPUs for the primary and backup CPU. You can change these specifications during execution so that a backup can be created or removed at any time. For more detailed information on designating the primary and backup CPU, see the *NET24-XPNET Network Control Operations Guide*.

Application process management

The XPNET process manages the application processes defined to it according to the attributes specified for each application process. Process attributes, such as the CPU in which to create the application process, the object file name, process name, process startup logic, and the priority assigned to the process are set and altered using network control commands. For more information on process attributes, see the *NET24-XPNET Network Control Command Reference Manual*.

Process startup

You can configure the XPNET process to start and manage each application process using one of the methods described in the following table:

Startup option	Description
Automatic	The XPNET process starts the application process automatically at XPNET node startup. If the application process fails, the XPNET process automatically restarts the application process up to a configurable maximum number of times. This prevents the XPNET process from looping on an attempt to restart a badly configured or continually failing process. The START PROCESS command and RED (Retry Evaluation Delay) attribute reset the failure count so that the process can again be started automatically by the XPNET process. AUTOPRI and WARMPRI attributes enable you to control the order in which the processes are started.
Demand	The XPNET process starts the application process automatically when the XPNET process receives a message for the process. If the application process fails or stops, the XPNET process does not restart the application process until it receives another message for the process.
Command	The XPNET process starts the application process when the XPNET process receives a START PROCESS command for the process. If the application process fails or stops, the XPNET process does not restart the application process unless the XPNET process receives another START PROCESS command for the process or a warmbackup node takeover occurs.

Startup messages

Each application process started by the XPNET process automatically receives the HP NonStop operating system startup system message followed by the XPNET proprietary startup message. Application processes written in source languages such as C++ and COBOL perform their startup in HP NonStop library code and must receive the HP NonStop operating system startup message for a valid startup to be initiated. For application processes that do not require the HP NonStop startup message, the message is ignored by the XPNET API runtime library (SKELB or XNLIB) being used by the application. In this way, XPNET is able to support applications developed in a variety of languages without making configuration changes for each process in the network.

Processes run as high PIN

The NET24-XPNET product supports the extended features of the HP NonStop operating system that enable a process to run with a high process identification number (PIN). A PIN is a unique, system-assigned identifier with a numeric value that identifies processes running in a CPU. The HP NonStop operating system allows up to 65,534 processes to run simultaneously in a single CPU. High PIN is defined as a process identification number in the range of 256 to 65,534. Low PIN refers to a process identification number in the range of 0 to 254. Both the XPNET process and the application processes controlled by the XPNET process can be started as either low or high PIN processes.

State machines

The XPNET process manages lines, links, processes, and stations using a state machine. The state machine defines all possible states associated with the object, the events that can occur during each state, and the state the object enters when the event occurs. The NCPI Server process manages the state machine for the XPNET process itself.

A state identifies the status of an object in relation to its readiness to do work. For example, processes have six possible states: Abnormal, Disabled, Started, Starting, Stopped, and Stopping. Each process is in one state at any given time.

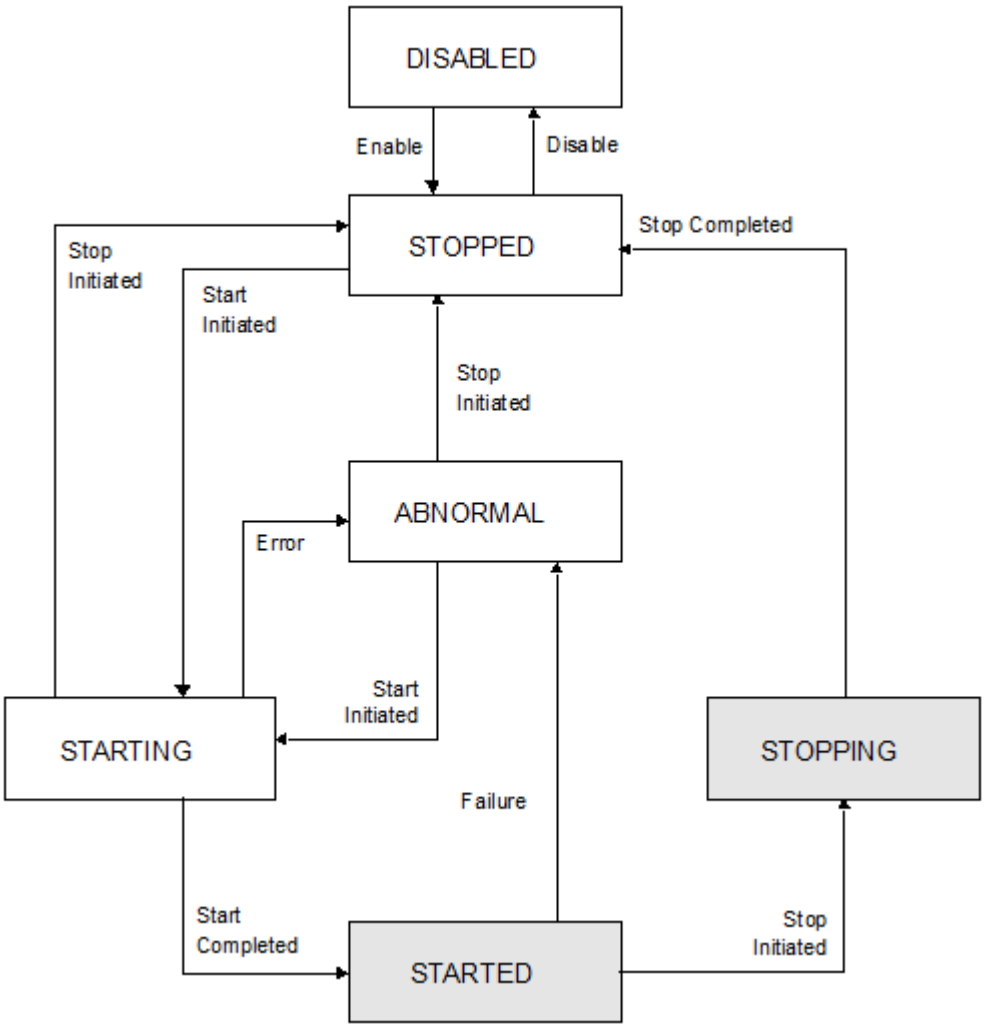
An event is a specific activity occurring in relation to the object. Each time an event occurs, the XPNET process determines the action to take and the new state the object is to enter.

For processes, the typical progression through these states is Disabled, Stopped, Starting, Started, Stopping, and Stopped. The Abnormal state occurs only if the process fails. The table below indicates how a process progresses from one state to another. Following the table is a graphic illustration of the state transitions for the process object type.

From state	To state	Event causing the transition
Abnormal	Starting	A START command was issued for the process.
	Stopped	A STOP command was issued for the process.
Disabled	Stopped	An ENABLE command was issued for the process or the process was added in the Enabled configuration state.
Started	Abnormal	The process failed. This transition occurs automatically.
	Stopping	A STOP command was issued for the process.
	Stopped	An ABORT command was issued for the process.
Starting	Abnormal	The process failed to start. This transition occurs automatically.
	Started	The process started successfully. This transition occurs automatically.
	Stopped	A STOP command was issued for the process.
Stopped	Starting	A START command was issued for the process.
	Disabled	A DISABLE command was issued for the process or a DELETE command was issued for the process while the process was in the Enabled configuration state.

From state	To state	Event causing the transition
Stopping	Stopped	The process completed its shutdown steps or an ABORT command was issued for the process.

The following illustration graphically depicts the state transitions described above for the process object type:



For more information on object states and state transitions, see the *NET24-XPNET Network Control Operations Guide*.

Memory management

The XPNET process uses extended memory for the following purposes:

- To store information used during processing (for example, information about the configuration of the objects that it manages).
- To store its destination table used for routing messages.
- To provide a dedicated input and output buffer for each process and line started by the XPNET process. When the XPNET process starts a process or line, the XPNET process allocates an area of extended memory equal

to the largest message the process or line can send or receive. When the process or line stops, the XPNET process de-allocates the extended memory buffer.

- To queue messages during processing. Queuing results when messages arrive at a rate faster than your application or device can process them. Other causes of queuing include temporary line outages, hardware failures, and software failures.
- To manage XPNET message payloads on behalf of processes that cannot receive messages with payloads. Payloads are areas of data in addition to the application data, which can be added to the XPNET message. Certain XPNET STATION or LINE protocols support sending payloads to the application. If a process cannot handle a message with payloads and such a message is queued to the process, XPNET can remove the payload from the message, save the payload in memory, and reattach the payload to the response which comes from the process. Refer to the Payload Processing section of the *NET24-XPNET Application Programming Reference Manual* for more details.

The size of the extended memory pool is user-configured. For more information on calculating the size of extended memory to use, see the *NET24-XPNET Network Control Operations Guide*.

The XPNET process supports a memory alert threshold so that you know when extended memory limitations are being reached. The memory alert threshold is a user-configured percentage of memory used above which the XPNET process begins generating event messages about the percentage of memory used. While this threshold is exceeded, the XPNET process evaluates memory usage at a user-defined interval and generates an event message about the percent of memory used at the end of each interval. For more information on the memory alert threshold, see the MAT attribute description in the *NET24-XPNET Network Control Command Reference Manual*.

Queue facilities

Each dest, link, process, and station defined to an XPNET process has a queue associated with it. When messages arrive at the XPNET process faster than a link, process, or station can accept them, the XPNET process queues the messages until the link, process, or station can accept them. Depending on circumstances, messages can queue at the link, process, or station, or at a corresponding dest entry in the destination table. Other causes of queuing include temporary line outages, hardware failures, and software failures.

Queue management

Queue management is a function of the XPNET process. The XPNET process keeps track of how many messages are queued to each dest, link, process, and station. You can view the number of messages queued to an object using network control commands. In addition, the NET24-XPNET product supports user-configured thresholds to make managing queues easier and to control the initiation of network overload management. These thresholds include a queue alert threshold, a queue management index, and a queue management threshold for each dest, link, process, and station.

Queue alert threshold

The queue alert threshold is a user-configured number of messages that causes the XPNET process to begin generating threshold event messages about the queue. You can specify a queue alert threshold separately for each dest, link, process, or station.

The current QAT percentage is passed to the application in the XPNET message header so it can participate in the QAT / NOM logic. As the application is aware of the types of messages (requests, responses, reversals, and so on), it can make intelligent decisions (start discarding requests, stand in) before it becomes a problem in the XPNET node.

The XPNET process generates an event message when the number of messages in the queue exceeds the queue alert threshold. In addition, the XPNET process starts a timer for a user-specified amount of time. When the timer expires, the XPNET process evaluates the queue.

A message is built and sent to the RTS clone process (if the RTS process is running and the RTS Flag is ON) when the object enters and exits the QAT state

If the number of messages in the queue exceeds the queue alert threshold, the XPNET process generates another event message and reinstates the timer. If the number of queued messages is less than the queue alert threshold, the XPNET process generates an event message indicating the queue no longer exceeds the queue alert threshold and does not reinstate the timer. For more information on the queue alert threshold, see the *NET24-XPNET Network Control Operations Guide*.

Queue management index and threshold

The queue management index and queue management threshold work together to regulate messages being added to an object queue in a queueing situation. These user-configured thresholds, used together or individually, determine when the XPNET process begins performing network overload management functions and can be set for each dest, link, process, and station.

The queue management index specifies the percentage of XPNET process total memory used before the XPNET process begins evaluating network overload management actions for the queue. The queue management threshold specifies the number of messages that can be queued to an object before the XPNET process begins evaluating network overload management actions for the queue. When both the queue management index and the queue management threshold are exceeded for a specific queue, the XPNET process generates an event message and begins taking network overload management actions.

A message is built and sent to the clone RTS process (if it is running and the RTS flag in ON) when the object enters and exits the NOM state.

When the XPNET process begins taking network overload management actions, the XPNET process evaluates each message that needs to be added to the queue and takes one of the following actions based on information in the message or node attribute settings:

- If the message header indicates the message should not be discarded, the XPNET process adds the message to the queue.
- If the message header indicates the message should be returned, the XPNET process returns the message to its source.
- If the XPNET node has been configured to save network overload management messages to disk, the message is written to a named extended memory segment. For more information on saving network overload management messages to disk, see the *NET24-XPNET Network Control Operations Guide*.
- Otherwise, the XPNET process discards the message. At a user-specified interval, the XPNET process generates an event message that indicates how many messages have been discarded.

The application process is responsible for setting flags in the message header. In addition, the XPNET process saves network overload management messages if the message originates from a station that has the NOMOVERRIDE attribute set to ON. Messages continue to be evaluated and processed as described above until the queue falls below the queue management threshold, the XPNET process memory utilization falls below the queue management index, or both. This configuration and state information is passed to the application in the XPNET message header so it can participate in the NOM logic, for example, discarding new requests to prevent stale responses or reversals.

For more information on the queue management index and threshold, see the *NET24-XPNET Network Control Operations Guide*.

Queue management commands

In addition to these automated queue management features, the NET24-XPNET product supports network control commands that enable an operator to transfer messages from one queue to another queue or to write messages from a queue to a file. After you have resolved the queueing problem, the messages in the file can be read back to the original destination queue or to a new destination queue. The network operator can also use commands to delete a specified number of messages from the queue or to delete all messages in the queue. These commands are documented in the *NET24-XPNET Network Control Command Reference Manual*.

Application process queue management

Individual application processes can also participate in network overload management functions. An application process can be written to make decisions about how to handle messages in a queueing situation. For example, an application process can allow messages in progress to complete, while at the same time preventing new messages from entering the system. As another example, an application process can be written to execute alternate logic for how to process the message if a queueing situation exists.

To participate in network overload management functions, an application process must be informed about the status of its queue. Before delivering a message to an application process, the XPNET process adds information to the message header informing the application process of its queueing situation. The XPNET process provides the information to the process about its queue. The information includes a queue alert threshold percent and a network overload management percent.

Queue alert threshold percent

The queue alert threshold percent indicates the number of messages in the queue in relation to the user-configured queue alert threshold for the application process. For example, if the process queue contains 160 messages waiting to be processed and the queue alert threshold for the process queue has been configured at 200, the queue alert threshold percent reported in the message header is 80 percent. The XPNET process updates this information in the header before delivering the message to the process. The application process can take appropriate actions if the queue is becoming critical as indicated by the queue alert threshold.

Network overload management percent

The network overload management percent indicates how close the XPNET process is to performing network overload management actions on the process queue. This percentage represents the lower of two percentages: the percentage of messages in the queue in relation to the queue management threshold for the application process or the percentage of memory used in relation to the queue management index for the application process.

Since both the queue management threshold and queue management index must be met or exceeded for the XPNET process to take network overload management actions, the lower of these two percentages gives an indication of how close the XPNET process is to taking action. The XPNET process updates this information in the header before delivering the message to the application process. The application process can take appropriate actions if the queue is becoming critical as indicated by the queue management threshold and the queue management index.

If the message was routed through a service, the message header also contains information about the current queue state for that service destination queue.

Message routing

The XPNET process routes messages to the destination defined within the message or to the destination configured for a station. The XPNET process routes messages to destinations on its local XPNET node by adding the message to that destination queue. If the message destination is external to the local XPNET node, the XPNET process queues the message to the link for the remote XPNET node.

If a destination cannot be found, the XPNET process saves the message in an unknown destination queue. The XPNET process periodically sends out discovery messages to discover the location of the unknown destination.

Subject to limits imposed by specific protocols, the XPNET process is capable of routing messages of up to 10 megabytes (10,485,760 bytes) when large message routing (also known as message chunking) is used. Setting the MAXLARGEMSG attribute to a value greater than 32,000 bytes activates large message routing. Without large message routing, the XPNET process is capable of routing messages of up to 32,000 bytes in length. These message lengths include the message header. Because of the performance issues created by extremely long

messages, ACI recommends that a practical limit of 1 megabyte be used. Additionally, warm-backup should be used if allowing large message routing.

The XPNET process can also use Routing Profile Configuration (RPC) routing to determine how to route incoming messages from a station or messages from a process. For information about RPC, refer to the *NET24-XPNET Routing Profile Configuration Guide*.

Message routing destinations

The XPNET process accomplishes default routing by taking all messages received from a particular station or process and routing them to the destination specified for that station or process. You can configure a default destination for each station and process managed by the XPNET process. Default destinations can also be changed online using network control commands. The destination specified can be a process, station, or service. A service is a function provided by one or more processes or stations.

When you specify a service as the default destination, the XPNET process invokes service-based routing. Each process and station in an XPNET system can be associated with one or more services. When the XPNET process receives a message with a service as the destination, the XPNET process delivers the message to a provider of the service. For more information on service-based routing, see the *NET24-XPNET Network Control Operations Guide*.

The use of content-based message routing (described below) can cause messages to be sent to destinations other than the default destination. The XPNET process uses the default destination if content-based message routing is not configured or no destination string match is found in the message when content-based message routing is used. In order for the XPNET process to perform default or content-based message routing for a message from a process, the symbolic destination must be set to blanks in the message header. For more information on default message routing, see the description of the DESTINATION attribute in the *NET24-XPNET Network Control Command Reference Manual*.

Destination string routing

Message routing can be defined within the content of the message itself. Content-based message routing seeks to match designated areas in the message against specified strings and, if a match is found, routes to a given destination.

You configure destination string routing at the device level. The device attributes define information that can be shared by stations or processes of similar type, thus reducing the amount of information needed for each station or process defined to an XPNET node.

The XPNET process supports a table of up to 32 destination string entries for each device. The table includes such information as the position within the message at which the content to be matched against is located, the actual character string to be compared to the message content, and the destination name. More complex routing can be configured for each device using a Routing Profile Configuration File (RPCONF). The XPNET process routes incoming messages to the destination specified in the destination table entry if the specified character string matches the appropriate string in the message from the station. For more information on destination string routing, see the *NET24-XPNET Network Control Operations Guide*.

Using an Integrated Endpoint process

Message routing can also be achieved with an Integrated Endpoint process. The Integrated Endpoint process is an object-oriented application process, developed under the integrated transaction services (ITS) architecture, that performs complex routing decisions. The Integrated Endpoint process is available for purchase as a separate product. For more detailed information on the BASE24 Integrated Endpoint process, see the *BASE24 IEP Configuration Manual* and the *BASE24 IEP Objects Reference Manual*.

Data communications

The XPNET process is responsible for the management of data communications facilities. Characteristics of a data communications line are defined to the XPNET process as station, line, and device attributes.

As part of its data communications support, the XPNET process is responsible for the following functions:

- Providing a communications link between application processes running on the HP NonStop computer and application processes running on another computer
- Providing a communications link with endpoint devices connected to the XPNET system
- Isolating application processes from protocol-specific requirements
- Controlling line use
- Monitoring station and communications line errors
- Governing responses to station and communications line errors according to user-configured attributes
- Controlling message synchronization

Specific types of computer systems and devices have specific communications requirements, and therefore require different protocols. For example, the protocol requirements for communicating with an external mainframe computer system are different than the protocol requirements for communicating with a point-of-sale device. Basic data communications functions are standard in the XPNET process. Specific protocol support is bound into the XPNET process as needed.

Data communications lines are opened and closed when either a fatal error condition requires this action or it is requested by a network operator. When a station is started on an idle line (no other stations are active on the line), the XPNET process opens the line. When the last station is stopped (all other stations are already stopped), the XPNET process closes the line.

Supported protocols

Following are types of protocols that NET24-XPNET supports:

- Bisync Multipoint Supervisor
- Bisync Multipoint Tributary
- Bisync Point-to-Point Primary
- Bisync Point-to-Point Secondary
- HTTP
- Multipoint Supervisor Burroughs
- LU 6.2
- MQSeries
- OSI
- Prestel-Bulk-Update
- SDLC XF Multipoint Supervisor
- SNA Primary
- SNA Secondary
- TCP/IP (TLS)
- TTY
- UDP/IP
- VISA
- WebGate Interface (WGI)
- X.21
- X.25

The NET24-XPNET product supports a communications application programming interface (API) that enables the development and support of regional- or customer-specific protocols. For more information on the communications API or support of specific protocols not listed here, contact your ACI account representative.

Event management facilities

The XPNET implementation of event management builds on the HP NonStop Event Management Service (EMS). The XPNET process generates tokenized event messages and sends them to EMS.

XPNET EMS implementation features

The XPNET process generates event messages that assist you in performing problem management by providing the information to detect, diagnose, and resolve hardware and software failures. Information is also provided on capacity and performance management so that you can monitor performance and usage of critical system resources.

The XPNET implementation of EMS provides the following features:

- An event message can be generated every time an object monitored by the XPNET process changes state.
- Event messages can be suppressed by object type and event message type at the XPNET node level to enable operators to select the event messages they want generated.
- Event messages generated by the XPNET process are fully tokenized.
- The ViewPoint event detail file, which contains cause and remedy information for each event message, is created programmatically at installation. You can access event detail file information online using EMSperus, an ACI-supplied event management utility.
- Each XPNET node can open a primary and an alternate collector process at startup.

Event message types from the XPNET process

The XPNET process generates an event message every time an object changes from one state to another as well as when other specified activities occur. Following are descriptions of the types of event messages the XPNET process generates:

Object unavailable	Generated when an object changes from a state in which it is considered available to a state in which it is considered unavailable.
Object available	Generated when an object progresses from a state in which it is considered unavailable to a state in which it is considered available.
Other state change	Generated when an object changes from a state in which it is considered available to another state in which it is still considered available, or when an object changes from a state in which it is considered unavailable to another state in which it is still considered unavailable.
Object crossing threshold level	Generated when an object crosses a user-defined threshold level.
Trace	Contains trace information for a line and selected system memory and resource utilization information.
Debug	Contains program debugging information.
Diagnostic	Contains diagnostic information about a problem or situation in the system.
Informational	Contains general information about the operation of the system.

Event messages suppression

The XPNET implementation of EMS enables you to limit the number of event messages that you see. Limiting the event messages that the XPNET process generates is referred to as suppressing event messages.

The XPNET process generates event messages when an object becomes available, unavailable, or undergoes an other state change; event messages that contain debug, diagnostic, and general information; and event messages when an object crosses a user-defined threshold. As a result, a large number of event messages can be generated. You can limit the number of event messages being generated by suppressing particular types of event messages. For each object type, you can specify whether the XPNET process should generate the event message types described in this document.

You can specify whether each category of event messages should be generated by using network control commands. For information on how to specify the generation of event messages, see the *NET24-XPNET Network Control Command Reference Manual*.

See *Network control* for information on the content of event log files. For more information on the XPNET implementation of EMS, refer to the *NET24-XPNET Event Management Manual*.

Audit facilities

The XPNET audit function enables system analysts and network operators to trace message traffic to and from the XPNET process. An audit of message traffic has the following purposes:

- Analyze message traffic
- Write messages to disk (providing nonvolatile storage of messages)
- Extract message selectively (using a user-written filter) from the audit file and moved to a QWRITE file, which can be re-introduced to an XPNET node.

Message traffic audit selection

An audit copies designated messages or chunks of a message between the XPNET process and application processes or stations to an audit file. You can also audit event messages and discarded (NOM'd) messages.

The audit function audits only input and output from those stations and processes designated by the operator. You can designate the individual stations and processes to be audited using network control commands. For each station or process, you can specify auditing for messages or chunks of a message to the XPNET process and messages or chunks of a message from the XPNET process. Application processes can also designate individual messages or chunks of a message to be audited by setting a bit in the message header.

In addition, you can enable auditing for all stations and processes in the XPNET node regardless of these individual settings. You can resume normal auditing where only designated stations and processes have their messages or message chunks audited with one network control command.

⚠ **CAUTION:** Do not enable auditing for all stations and processes in a production network unless ACI instructs you to do so.

Audit files

The number of audit files used by the XPNET depends upon whether you have set up the internal audit process (AUDITIO is set to BUFFERED, NOWAIT, or WAIT) or the external audit process (AUDITIO is set to EXTERNAL).

Internal audit process

You can configure up to three audit files for the XPNET process to use. Files can be named using any network control facility. Audit files can be configured as disk files, or, if required, one or more audit files can be configured as a process. Initiation and termination of an audit is achieved through the use of network control commands.

When an audit is started, the XPNET process begins using the first audit file (or process) as the current audit file. This file is the current file until any of the following events occur:

- The file becomes full.
- A fatal error occurs on the file.
- A network operator requests a switch to a new audit file.
- Auditing is stopped and restarted.
- The XPNET node is stopped and restarted.

When any of these occur, the XPNET process attempts to switch to a new audit file. If you have configured two or three files, file usage rotates around the files continuously. Any records in an audit file when the XPNET process begins using the file are deleted.

For information on how the XPNET process determines the current audit file; how audit files rotate in response to an audit switch; and the effect of naming one, two, or three audit files, see the *NET24-XPNET Network Control Operations Guide*.

External audit process

When the audit process reads the message, it takes various actions based on the NODE configuration settings.

- If EXTAUDIO is ALWAYS then the message will be written to the audit file. The external audit process will create an indexed audit file (like how NOM files are managed in the XPNET node).
- If EXTAUDTCPIP is ON then the external audit process will also stream audit records on up to three TCP/IP connections based on its PROCESS configuration in the NEF. This works in conjunction with EXTAUDIO.
- If EXTAUDTCPIP is ON and EXTAUDIO is BACKUP, then audit records are only written to disk (AUDnnnnn) if there are no established TCP/IP connections. If one or more are established, then the audit records are sent over the TCP/IP connections (same audit record duplicated on each established connection) and not persisted to disk.
- If EXTAUDTCPIP is ON and EXTAUDIO is NONE, then audit records are not written to disk. If one or more are established, then the audit records are sent over the TCP/IP connections (same audit record duplicated on each established connection). If there are no connections, then the audit records are dropped and lost.

For information on how the XPNET process creates indexed audit files and routes messages through TCP/IP, see the *NET24-XPNET Network Control Operations Guide*.

Audit buffering

You can configure the buffering action for audited messages at the node level. Some degradation in system performance that can occur as a result of auditing can be mitigated by the type of buffering under which the audit operates. The XPNET process supports the following options for audit buffering:

Buffered Indicates that audited information should be stored in the audit buffer and written to the audit file when the buffer is full. The audit buffer for disk audit files is 56,000 bytes. The audit buffer for process audit files is 32,000 bytes. When the buffer is full, the XPNET process writes the information to the audit file using the operating system NOWAIT mode. This method of auditing has the least impact to system performance, but can result in the loss of up to 56,000 bytes of audited information in the event of a system failure. This option also prevents the ability to do REALTIME auditing.

External Indicates that the audited information that would normally be written to AUDITA, AUDITB or AUDITC by the XPNET node process are instead sent to the External Audit process via the internal #AUDIT-SERVICE dynamic service. System performance can be enhanced by using an external service to handle auditing.

No wait Indicates that audited information should be written to the audit file using the operating system NOWAIT mode. When the XPNET process receives a message to be audited, it writes the message to the audit file and continues processing. If the XPNET process receives subsequent audited messages before the write to the audit file completes, the XPNET process stores the messages in the

audit buffer. When the write completes, the XPNET process writes the next message in the buffer to the audit file.

Wait Indicates that audited information should be written to the audit file using the operating system WAIT mode. The XPNET process writes information to the audit file as it receives information to be audited. The XPNET process waits for the write to complete before doing any other processing.

Because the XPNET process stops processing until each write to the audit file completes, the WAIT mode can have severe impacts on system throughput. Use of this mode should be carefully evaluated by the system manager before the mode is implemented.

Audit file reading utility

The NETADRD utility is the XPNET system utility used to read an audit file. This utility can produce various types of output from an audit file. The output can be sent to a terminal or printed to analyze the captured messages. The output can also be written to a disk file. Messages in the disk file can be reintroduced to the XPNET system. See *Utilities* for an overview of the NETADRD utility. The NETADRD utility is described in detail in the *NET24-XPNET Network Control Operations Guide*.

Debugging facilities

Several debugging facilities are included in the NET24-XPNET product. Debugging facilities are an important tool for application developers. In addition, debugging facilities provide valuable problem solving tools for system analysts. An XPNET system provides debugging facilities in the following ways:

- Event messages that provide debug, diagnostic, or historical information
- Audits
- TRACE LINE command
- TRACE NODE command
- TRACE NODE SSL command
- Debug mode for a process
- Process information captures
- #DIAG-DEST internal diagnostic destination

Event messages

The XPNET process uses the HP NonStop Event Management Service (EMS) product as a logging facility. Event message types generated by the XPNET process are helpful in debugging include debug, diagnostic, and informational event message types.

Debug event messages include information about an object for which XPNET detects a problem. Debug event messages inform the analyst of the location in the program code where the event message was generated, thus providing a means to pinpoint the exact location in the code where the problem occurred.

Diagnostic event messages contain information about a problem or situation in the system. Like the debug event messages, the diagnostic event messages inform the analyst of the location in the program code where the event message was generated.

Informational event messages provide a historical view of activity at the object level. An analyst can use these event messages to discern events leading up to the occurrence of a problem.

For more information on the XPNET implementation of EMS, see the *NET24-XPNET Event Management Manual*.

Audits

The XPNET audit function enables system analysts and network operators to trace message traffic to and from the XPNET process. An audit can be run so that message traffic can be analyzed if the system is experiencing problems.

The XPNET process is responsible for routing all messages that flow through an XPNET system. An audit copies messages between the XPNET process and designated processes and stations, as well as individual messages identified by application processes, to an audit file. Because the XPNET process handles all message flows, the image of a message can be easily captured.

The NETADRD utility is used to produce various types of output from the audit file. The output can be read from a terminal or printed to analyze the captured messages.

For more information on auditing, see the *NET24-XPNET Network Control Operations Guide*.

TRACE LINE command

The TRACE LINE command initiates a trace of a line object. This command can be used in one of two ways.

- The TRACE LINE command can be used to produce a real-time, continuous trace of a line. Trace information for the line or lines is sent to EMS in a format that is specific to the protocol of the line being traced and continues to be sent until a command is issued to disable the trace.
- The TRACE LINE command can also be used to produce a dump of the line event stack. The entire event stack is sent to EMS and trace event messages are printed in reverse chronological order. This is not a continuous trace of line events. It is a snapshot of the line events at the time the trace is taken. After the line event trace is produced, the trace is automatically disabled.

For detailed information on how to use of the TRACE LINE object management command, see the *NET24-XPNET Network Control Command Reference Manual*.

TRACE NODE command

The TRACE NODE command initiates a retrieval of network trace information currently available from the internal trace table of the node. With this command, a file is specified to hold trace information. After the command is implemented, the trace file can be sent to ACI Worldwide for analysis. NCPCOM also has the ability to display the trace information.

For detailed information on the use of the TRACE NODE object management command, see the *NET24-XPNET Network Control Command Reference Manual*.

TRACE NODE SSL command

The TRACE NODE SSL command initiates the retrieval of SSL available from the internal trace table of the node. With this command, a file is specified to hold trace information. After the command is implemented, the trace file can be sent to ACI Worldwide for analysis. For detailed information on the use of the TRACE NODE SSL object management command, see the *XPNET Internal TCP/IP Protocol Implementation Guide*.

Debug option

Use the DEBUG process attribute to start an application process in debug mode. This is a useful, real-time, problem analysis tool for customer-written application processes that are experiencing execution problems. The debug option is most often used in test systems. This attribute gives the network operator access to HP NonStop interactive DEBUG or Inspect commands from the time the process initializes, before any of the process code is executed. If a process is started in DEBUG mode, the home terminal of the process receives the HP NonStop DEBUG or Inspect prompt. A home terminal for the process can be specified with network control commands.

Process information

Use information about a process at the moment it encounters processing difficulties to determine the cause of the problem and to help identify potential solutions to the problem. Capturing process information gives you access to information such as the values of program variables and the contents of memory at the moment the process encountered the problem.

Process snapshot files and the XPNET process

If the XPNET process abends, a standard saveabend file is always generated. XPNET saveabend files must be debugged by ACI personnel; however, these files can be used in certain cases by the NETDMPRD utility to introduce queued messages back into a running XPNET process. See the *NET24-XPNET Network Control Operations Guide* for details about the NETDMPRD utility.

Process snapshot files and XPNET application processes

Applications that are compiled for XPNET Release 3.1 or newer and use either the Release 3.1 or newer SKELB or Release 3.1 or newer XNLIB libraries receive a standard saveabend file on abend as long as the SAVEABEND attribute of the process is set to ON (the default value). ACI highly recommends having this attribute set to ON. If this attribute is set to OFF, if may not be possible to provide support if the application abnormally terminates. Standard HP tools are used to view the saveabend file.

#DIAG-DEST internal diagnostic destination

In certain situations during message processing, a message found to be formatted poorly, such that the XPNET process cannot determine how to handle it, can be forwarded to the #DIAG-DEST internal destination within the node. A limited number of messages are retained in the #DIAG-DEST queue for later perusal. Several ways of capturing or viewing these messages are available. See the *NET24-XPNET Network Control Operations Guide* for additional information.

Application programming interfaces

Application programs perform application-specific processing to meet the needs of your unique business environment. The NET24-XPNET product supplies application programming interfaces (API) enabling the development of application programs that access the services provided by the XPNET process.

Because the NET24-XPNET product is designed to achieve a separation of responsibilities along functional lines, application development is isolated from many functions handled by the XPNET process. For example, support for specific protocols is bound into the XPNET process making the application essentially independent of communications protocols. In addition, applications need not be designed for continuous processing. The XPNET process contains logic for continuous processing.

There are two application programming interfaces standard with the NET24-XPNET product. Each interface supports application development in different programming languages.

- The BASE24 SKEL application programming interface can be used to write procedural application programs using the TAL language. Information on this interface is available in the *BASE24 Application Programming Reference Manual*.
- The Network Library (XNLIB) application programming interface can be used to write application programs in TAL, C, C++, or COBOL. The NETLIB interface also supports the generation of fully tokenized EMS event messages. Information on this interface is available in the *NET24-XPNET Application Programming Reference Manual*.

General information that is common to developing applications using either the BASE24 SKEL interface or the XNLIB interface can be found in the *NET24-XPNET Application Programming Guide*.

Section 3: Network control

Network control is a broad term encompassing all aspects of keeping an XPNET system operating efficiently according to the policies and operating procedures of an organization. This includes such functions as monitoring the general operation of the network; keeping the various lines up and stations running; starting and stopping processes, nodes, and links; and adding and reconfiguring dests, devices, lines, links, nodes, processes, and stations.

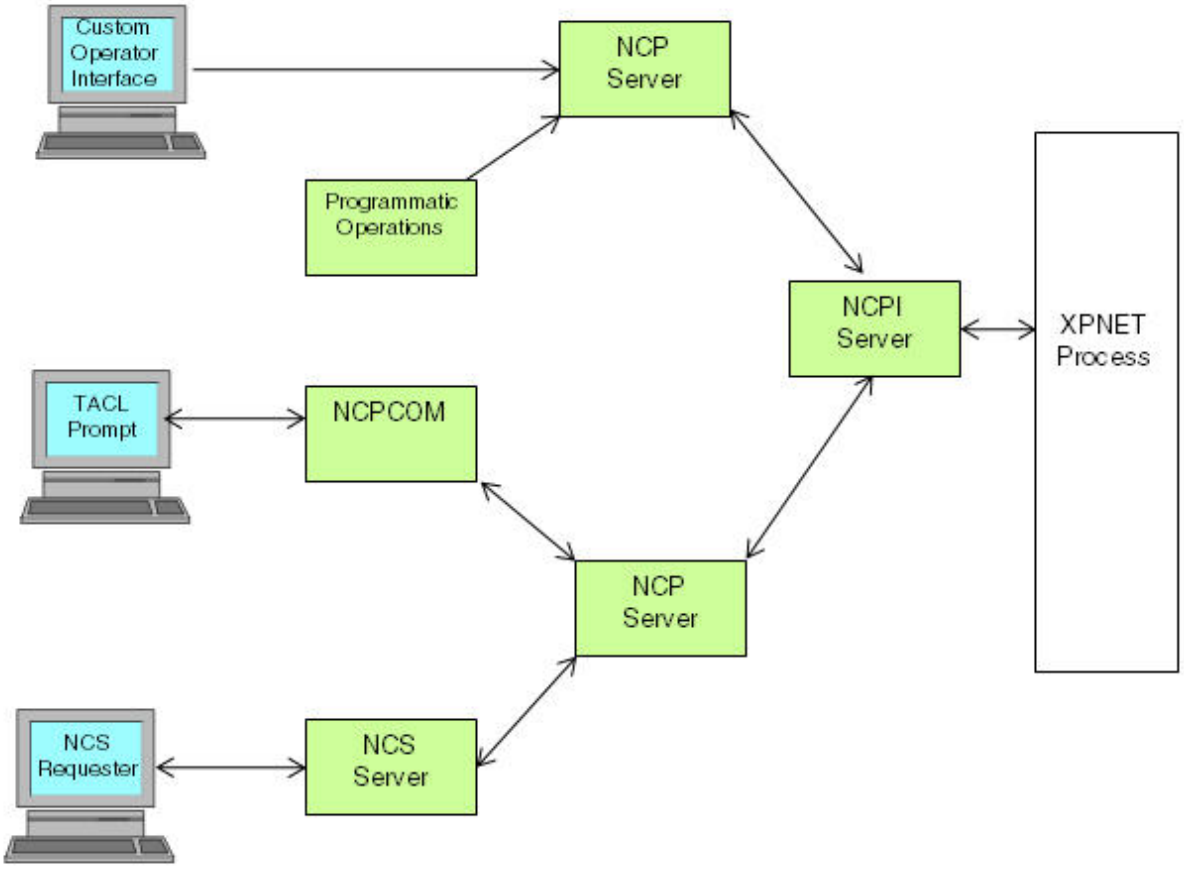
The basic tools of network control consist of operator interfaces, network control commands issued from these operator interfaces, event log files, and security features. These tools enable operators to effectively monitor and make online modifications to the XPNET system.

The network control environment

In order to control aspects of an XPNET system, the network operator must have the ability to communicate with the XPNET process. A set of operational interface processes, common to all XPNET systems, are included in the NET24-XPNET product to achieve this. These processes enable messages that control the operation of the XPNET system to pass between the XPNET process and an HP NonStop monitor. The XPNET system can communicate with or without the HP NonStop Pathway system.

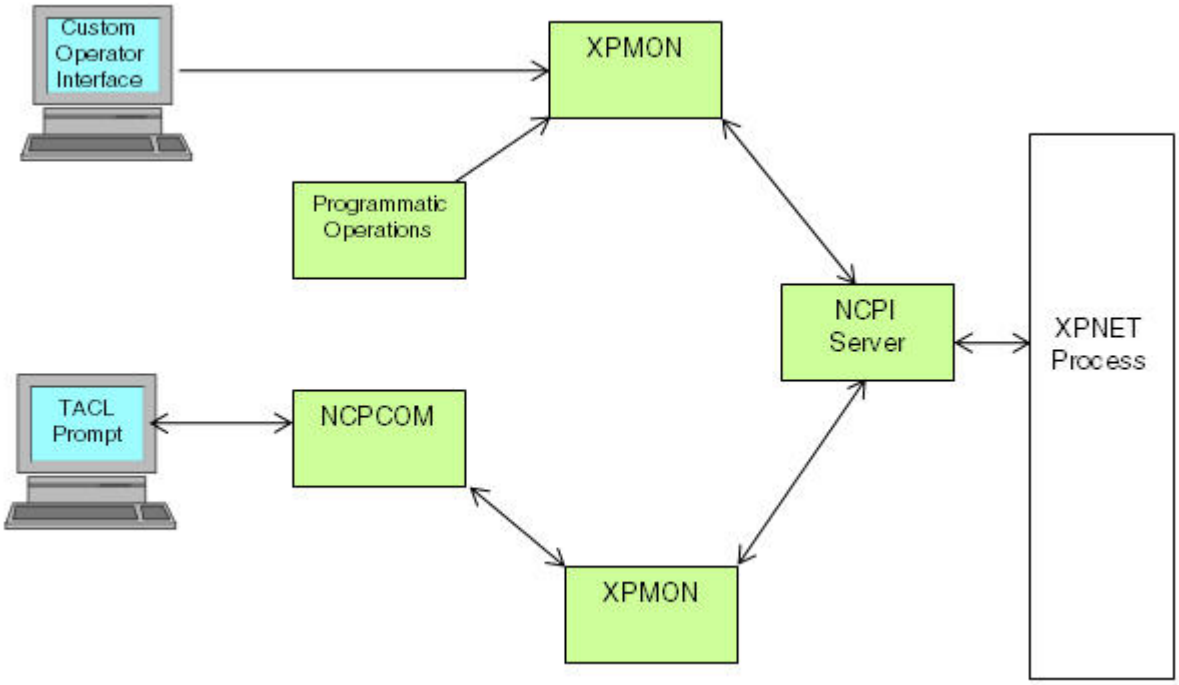
XPNET operational interface processes include the Network Control Supervisor (NCS) Server process (which is used in a Pathway environment only), the Network Control Point (NCP) Server process, and the Network Control Point Interface (NCPI) Server process. These processes are illustrated in the following graphic. Together these components provide a command set and interface commonly used for network control of system objects.

The following illustration shows four operator interfaces from which commands are sent to the XPNET process in a Pathway environment.



The Network Control Supervisor (NCS) screen provides a formatted interface for entering network control commands. The NCS Server process interprets commands issued from the NCS screen. The Network Control Point Communications (NCPCOM) utility provides an interactive prompt for entering network control commands. In addition to these standard interfaces, you can create custom operator or programmatic interfaces. Standard and custom interfaces are described in more detail in the topic “Operator Interfaces.”

The following illustrates the interaction without the Pathway system.



Network control commands issued from operator or programmatic interfaces are processed by the XPMON process, which is the control point for all network commands in an environment without Pathway.

After the NCPCOM process validates a command for syntax, it sends the command to the XPMON process, which determines to which Nodes (NCPI) Server process sends the command, which validates security and controls message flow between the XPMON process and the XPNET process. The NCPI Server, NCP Server, and NCS Server processes are described below.

NCPI Server process

The Network Control Point Interface (NCPI) Server process provides the programmatic interface to an XPNET node. It performs all security processing. There is one NCPI Server process per XPNET node.

The NCPI Server process is responsible for validating operator command security. This includes commands originating from all of the following sources:

- NCS screen (Pathway environment only)
- NCPCOM utility
- Custom operator interfaces
- Custom programmatic interfaces
- HP NonStop or third-party developed programmatic interfaces (such as NonStop NET/MASTER or Programmatic Network Administrator)

If PARAM ENABLE-SECURITY is set to ON or DETAIL, the NCPI Server process accesses security files to determine if the operator issuing the command has a valid user identification and password and if the operator is granted access to the command issued and the XPNET node involved in the command.

NCP Server process

The Network Control Point (NCP) Server process determines the appropriate NCPI Server process to receive a command. The NCP Server process receives commands from any operator interface and determines the XPNET node involved in the command. The NCP Server process then forwards the command to the NCPI Server process for that XPNET node.

If the XPNET system spans multiple HP NonStop nodes, if configured to do so, the NCP server forwards commands to a remote copy of the NCP server, which in turn sends the commands to the appropriate local NCPI processes or nodes.

NCS Server process

The Network Control Supervisor (NCS) Server process is a Pathway server that interprets network control commands coming from the Network Control Supervisor (NCS) screen. This is used only if Pathway is licensed.

Operator interfaces

Operator interfaces enable the network operator to issue commands that control an XPNET system. The network control subsystem provides a command and control interface to an XPNET system. Standard interfaces include a formatted screen or an interactive prompt.

In addition to network control facilities, the X.25 Network Inquiry (XNI) subsystem enables operators to send supervisory inquiries into the DATAPAC RAPID service in Canada. Network control facilities and the XNI subsystem are described below.

Network control facilities

Two tools are available for controlling the network: the Network Control Point Communications (NCPCOM) utility and the Network Control Supervisor (NCS) screen. Users also can develop their own custom operator or programmatic interface using the management programming interface. From any of these interfaces, an operator can issue inquiries and commands to the network.

NCPCOM utility

The NCPCOM utility provides an interactive prompt for entering commands. You start the NCPCOM utility by issuing a RUN command from the TACL prompt.

The first command you must enter in any NCPCOM utility session is the PATH or GATE command. PATH is used in a Pathway environment to establish a connection to the Pathway system that interfaces with the XPNET process. GATE is used when Pathway is not licensed from HP to establish a connection to the XPMON process that interfaces with the NCPI server process. Additionally, if necessary, you can enter the command that logs you on as a specific user. This command is required if security is enabled. If you do not enter these commands at the beginning of an NCPCOM utility session, all commands fail due to no connection to an XPNET system or a security error.

The NCPCOM utility provides for setting a default HP NonStop system, node, object type, and object name for a series of commands, limiting the number of keystrokes required to issue a command. This is achieved using the ASSUME command, which establishes the default value for the HP NonStop system, node, object type, and object name for all subsequent commands. A sample NCPCOM prompt is shown below:

```
NODE 8 >
```

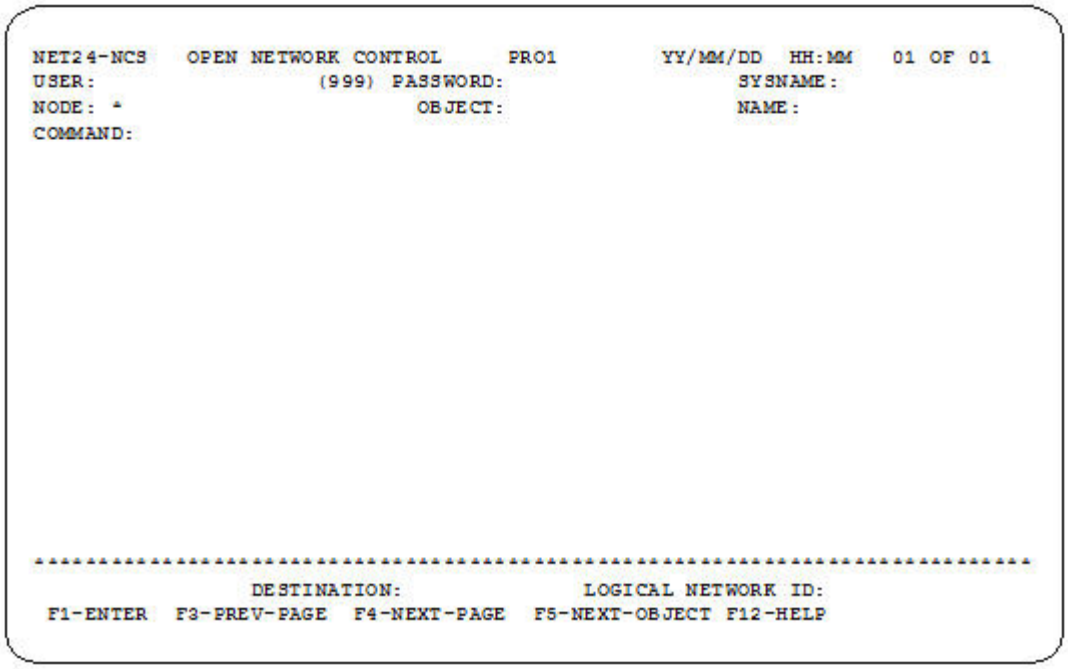
The sample prompt indicates that the default object type is NODE and that 7 commands have been entered previously.

The NCPCOM utility maintains a buffer of the last twenty commands; responses are not included. The buffer can be displayed using the HISTORY command and can be used to re-execute any command.

All commands can be entered from the NCPCOM prompt. See *Network control commands* for an overview of available network control commands. Security features of the operator interfaces are described below. For more detailed information on the NCPCOM utility, see the *NET24-XPNET Network Control Operations Guide*.

NCS screen

The NCS screen provides a formatted interface for entering network control commands. This screen is used in a Pathway environment only. The NCS screen is accessed through a logon screen and a series of menus. To issue a command once the NCS screen is displayed, you enter information in the applicable fields on the screen and press a function key. The command is then interpreted by the NCS Server process sent to the NCP Server, which forwards the commands to be processed by the NCPI Server process. Following is an example of the NCS screen.



The formatted interface helps you to complete a command by building command elements from the information entered in the screen fields. The fields presented to accept user input include the user name and password, the HP NonStop system name, the XPNET node that the command is to be executed against, the object type, the symbolic name of the object that is to be referenced by the command, and the command keyword.

The NCS screen is a flexible interface. You can use the screen fields for input or type the entire command on the command keyword line of the screen. In addition, the NCS screen provides for setting a default HP NonStop system, node, object type, and object name for a series of commands, limiting the number of keystrokes required to issue a command. The NCS screen also maintains a buffer of the last ten screens of commands and responses. The buffer can be perused using function keys and can be used to re-execute any command.

The NCS screen can be used to issue most network control commands. See *Network control commands* for an overview of available network control commands. Security features of the operator interfaces are described below. For more detailed information on the NCS screen, see the *NET24-XPNET Network Control Operations Guide*.

Management programming interface

The XPNET network control facility defines the network control interface for XPNET systems. The external portion of this interface is documented, enabling you to develop custom operator interfaces, custom programmatic interfaces, or enabling the use of programmatic interfaces developed by third-party suppliers. Command syntax for custom operator interfaces and programmatic interfaces is based on the HP NonStop command language standard and is similar to other HP NonStop command interfaces. Commands share the same format whether they are entered by a network operator or initiated programmatically.

The *NET24-XPNET Network Control Operations Guide* contains information for developing custom and programmatic interfaces.

X.25 Network Inquiry subsystem screen

The X.25 Network Inquiry (XNI) subsystem screen enables operators to send supervisory inquiries into the DATAPAC RAPID service in Canada. Using the X.25 Network Inquiry subsystem with RAPID, you can send RAPID READ messages on selected user circuits. Statistical data received in response to the RAPID READ message is then formatted and displayed on the XNI subsystem screen. For more information on the X.25 Network Inquiry subsystem, see the *NET24-XPNET Network Control Operations Guide*.

Security and operator interfaces

Operator access to network control commands can be configured when security is enabled. Command access limitations enable you to specify for each user whether the user has access to each network control command. Command auditing creates an audit trail of network control command activity. Command security is the same whether commands are issued from the NCS screen or the NCPCOM utility. Security is discussed in more detail in *Network Control Security*.

Network control commands

The XPNET process receives control information from the operator through network control commands. The XPNET process acts on commands in a single-thread fashion (that is, until the XPNET process has completed processing of one command, another command cannot be accepted). When the XPNET process has completed the requested action, it returns a response to the requester of the service.

Network control commands are grouped according to their purpose into the following three categories: environment commands, inquiry commands, and object management commands. Each of these categories is described below.

Environment commands

Environment commands are operational and environmental commands that impact the network control environment. The following are environment commands:

ALLOW	Specifies the number of errors and warnings allowed.
ASSUME	Sets the default object type and object name.
COMMENT	Adds a comment to an obey file.
DELAY	Sets a delay before executing the next command in an obey file.
ENV	Displays the default object type, object name, XPNET node, HP NonStop system, and environment settings.
EXIT	Exits from the Network Control Point Communications (NCPCOM) utility.
FC	Retrieves a command from the NCPCOM command buffer for editing.
GATE	Specifies the PPD name of the XPMON process being used.
GETVERSION	Identifies the NCP Server process version.
HELP	Displays syntax information for network commands.
HISTORY	Displays the list of commands in the NCPCOM command buffer.
LOG	Initiates logging of commands and responses to a log file.
OBEY	Executes the commands in an obey file.
PATH	Specifies the PPD name of the Pathway system being used and the server class to which the NCP Server process is assigned.

RESET	Resets the values in the working set of attributes for the specified object type to their default values.
SET	Defines the working set of attributes for the specified object type.
SET ADDTYPE	Specifies the default configuration state to be in effect for subsequent ADD commands.
SET ALTERTYPE	Specifies the default configuration state filter to be in effect for subsequent ALTER commands.
SET DELETETYPE	Specifies the default configuration state filter to be in effect for subsequent DELETE commands.
SET INFOTYPE	pecifies the default configuration state filter to be in effect for subsequent INFO commands.
SET NCPTIMER	Specifies the length of time the NCP Server process waits for a response from the NCPI Server process.
SET PASSWORD	Modifies the password assigned to the current user.
SET PROMPT	Change command prompt.
SET SYNTAXONLY	Specifies that commands be processed for syntax verification only.
SET USER	Specifies the current user for an NCPCOM session.
SHOW	Displays the values of the working set of attributes for the specified object type.
TIME	Displays current time.
!	Re-executes a specific command.

See the *NET24-XPNET Network Control Command Reference Manual* for a complete description of the network control commands.

Inquiry commands

Inquiry commands display object and attribute settings for objects in the network. Each of these commands returns formatted information about objects in the network or about control settings used by objects in the network. The following are inquiry commands:

BSISTATUS	Displays information on the status of SBA/BSI services.
HINFO	Displays a hybrid of settings information about the object, combining several commands into one.
HSTAT	Displays a hybrid of statistics information about the object, combining several commands into one.
INFO	Displays current settings for attributes of the object.
LISTOBJECTS	Displays a list of objects that match specified criteria.
NEXTOBJ	Displays next object.
STATISTICS	Displays statistical information about the execution of the object.
STATUS	Displays information reflecting the current state of the object.

See the *NET24-XPNET Network Control Command Reference Manual* for a complete description of the network control commands.

Object management commands

Object management commands are used by the network operator to change the attribute or directive settings for an object and to control the network. The following are object management commands:

ABORT	Initiates shutdown of objects.
ADD	Adds objects to the system.
ALTER	Changes the values of configured attributes of objects.
CONTROL	Directs a specific action to be taken for objects.
DELETE	Deletes objects from the system.
DELIVER	Delivers a text string to objects.
DISABLE	Transitions the object from the Stopped state to the Disabled state.
ENABLE	Transitions the object from the Disabled state to the Stopped state.
KILL	Issues an immediate stop of the object.
RESETSTATS	Sets the execution history of objects to their null or initial value.
START	Initiates startup of objects.
STOP	Initiates shutdown of objects.
SWITCH	Switches processing from the primary XPNET process to the backup XPNET process.
TELL	Sends a string of text characters to an external object.
TRACE	Initiates a trace of the object specified.

See the *NET24-XPNET Network Control Command Reference Manual* for a complete description of the network control commands.

Network control security

Network control command security is configurable. At its most basic level, access to network control commands is determined by access to a network control facility. If a user has access to a network control facility, then, without implementation of further security measures, he or she can execute any command. Two additional security measures can be taken, either separately or in conjunction with each other. These measures are to limit network control command access by command and to audit network control commands.

Command access limitations

Command access limitations enable you to specify for each user whether the user has access to each network control command. When you have enabled command access limitations, you assign a command profile to each user in his or her security record. The command profile indicates, for each network control command, whether the user has access to the network control command.

You enable command access limitation during configuration of the NCPI Server processes. In addition, two files are required to limit network control command access. These files are the Network Control Security Profile File (NCSP) and the Network Control Security Specification File (NCSS). Command access limitations are specified on the security screen that updates the NCSP. Options include the following:

- A = Audit access
- C = CPCONF (XML based file to enable additional control of DELIVER command)
- N = No access
- U = Unrestricted
- V = View unrestricted (with auditing)
- Y = Access

The Command Profile Configuration Generator (CPCGEN) is used for CPCONF creation.

An operator's security records are associated with a specific command profile on the security screen that updates the NCSS.

Command auditing

Command auditing creates an audit trail of command activity. When you enable auditing by itself (without command access limitations), the XPNET process generates an event message each time someone executes a network control command. The event message identifies the user who issued the command, the XPNET node affected, the terminal the command was issued from, and the command that was performed.

You enable command auditing for nodes during configuration of the NCPI Server processes. This is used in conjunction with the command access limitations configured in the NCSP.

Access limitations and auditing commands

When you enable both command auditing and command access, the command profile controls not only whether the user can perform the command, but also whether execution of the command is audited. In this way, system administrators can audit selected commands or users. For example, new network control operators could be assigned a special command profile that gives audited access to some or all commands. Or, command profiles could be created that enable users to execute inquiry commands without being audited, while commands that change the system configuration are audited.

For more information on network control command security, see the *NET24-XPNET Network Control Operations Guide*.

Network Control Security Profile File

The Network Control Security Profile File (NCSP) enables you to set up network control command profile records. The command profile records indicate whether each network control command can be executed and whether the command is audited when it is executed. After you have created a command profile and stored it in the NCSP, you can associate it with the security record for one or more users.

Network Control Security Specification File

The security screen that accesses the Network Control Security Specification File (NCSS) enables you to specify the network control command profile to be associated with a user's security record.

The NCSS enables a single command profile to be assigned to multiple users. The command profile must be added to the NCSP before the profile can be assigned to a user.

The user's security record contains the following:

- User alias
- Node name
- Command profile name
- Password
- Network control access limitations by time of day, day of the week, and last date allowed
- Password requirements for minimum and maximum length, number of alpha and numeric characters, frequency of password changes, password reuse limitations, and maximum invalid password attempts allowed

Event log files

Event log files provide useful information about the system environment. Information maintained in the event log file can help you to do the following:

- Monitor events in your system as they occur
- Analyze past events

- Detect potential system problems
- Monitor object queues

Various application subsystems in an HP NonStop system generate event messages and send them to a collector process. The EMS collector processes are responsible for receiving event messages from HP NonStop and application subsystems (such as the XPNET system) and writing the messages to the EMS event log files. Event log files provide central locations where event messages are stored. Once event messages have been written to the event log file, the EMS distributor processes determine where each message should be routed. EMS can distribute event messages to processes, display devices, files, and collectors on other HP NonStop systems. EMS distributor processes can also access archived event log files in order to resolve problems, monitor events, or view old event messages.

Filters can be used to tailor the messages appearing in the event log of the EMS environment to meet particular logging requirements by enabling specific event messages to be handled in different ways. Event messages can be filtered by a variety of parameters including process, event number, subsystem ID, and subject. Users can select the event messages important to their specific requirements, while eliminating less important event messages. ACI provides filter source statements for a set of 25 sample filters.

Event log files viewing

ACI has developed four utility programs to enhance the functionality of EMS when it is being used by ACI products. You can use these utilities to assist you in performing daily activities required to maintain and monitor EMS.

The EMSdcom utility enables you to display the status of one or all EMS distributor processes, and to control a printing or forwarding distributor process through the entry of online commands. Online command options include such functions as changing the filter file used by the distributor, connecting to a specific event log file, adding or deleting files from the list of output locations for a printing distributor, and positioning the distributor to a specific log time.

The EMSperus utility provides a method by which current or archived event messages can be perused. Current event messages can be displayed in real time (that is, continuously as they are received by the collector process). All event messages that meet the current filtering requirements are displayed. The EMSperus utility facilitates such operator actions as customizing the format in which event messages are displayed, specifying what information is displayed in the event message, controlling the number of event messages displayed, and searching for the text of a specific event message.

The Logdater utility organizes event log files into daily subvolumes, enabling you to easily locate event messages from previous days of network operation. The Logdater utility cuts over to a new subvolume based on a calendar day. Each day at midnight, the utility automatically sends a message to the EMS collector process to close the event log file for the current day and to start logging messages to the event log file for the next day. Logdater utility cutover is separate from cutover or end-of-day processing of any application subsystems.

The Sudoterm utility provides the ability to capture COBOL library errors and home terminal messages that would otherwise be lost. Messages sent to the home terminal are generally not log messages but rather error messages. These error messages can be important in identifying errors occurring in an application, but they are usually displayed only on the home terminal for the application. By specifying the Sudoterm utility as the home terminal for an application, these messages are captured, partially tokenized, and sent to EMS. They become part of the event log file and can be used to analyze problems with the application.

ACI recommends using HP's Virtual Hometerm Subsystem (VHS) as the preferred replacement for the Sudoterm utility on the HP NonStop NS-series platform because the Sudoterm utility is not supported on that platform.

Event message content

The information displayed for each event message can be tailored to meet your site-specific requirements using the EMSperus utility. Certain information is available for display for all XPNET event messages. The information common to all XPNET event messages include such items as the identification of the subsystem that generated the event message (for XPNET applications, the XPNET subsystem identification), the subject with which the

event is concerned (for an XPNET object, the symbolic name of the object), an event number that corresponds with a category of event message types (these categories include the informational, diagnostic, debug, trace, object state transitions, and object-crossing-threshold event message types described in *Event message types generated by the XPNET process*), and the name of the XPNET process that generated the event.

Event detail information is also available to assist your analysis of a problem. Event detail information contains the probable causes and recommended actions for many of the event messages generated. In addition to the HP NonStop event detail file, separate event detail files can exist for each application subsystem. ACI supplies template source files that provide information about each event message generated by network-level processes. You can edit the template source files to provide additional site-specific information. Display of event detail information is set using the EMSperus utility.

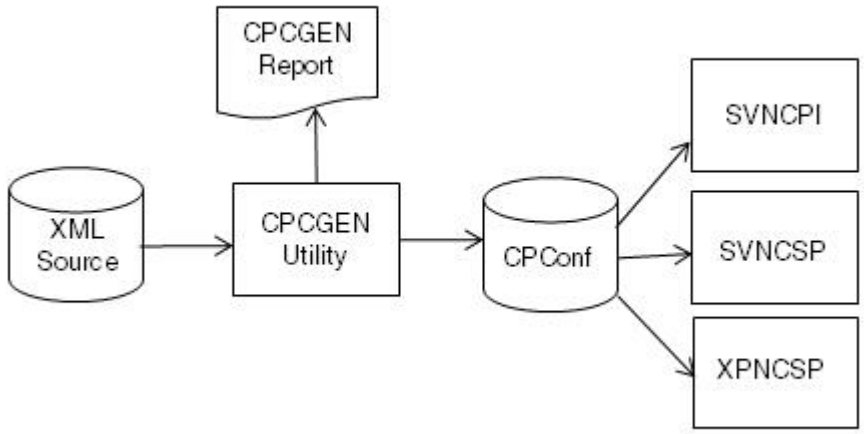
For additional information on the XPNET implementation of EMS, see the *NET24-XPNET Event Management Manual*. For additional information on EMS, see the HP NonStop *EMS Manual*.

Section 4: **Utilities**

Utility programs facilitate routine processing for network operators. They are usually dedicated to performing a specific task or a group of related tasks such as those involved in system configuration or system maintenance. The NET24-XPNET product includes several utilities to assist in the operation of the XPNET system.

Command Profile Configuration Generator (CPCGEN) utility

The CPCGEN utility is used to generate the CPConf from the XML source.

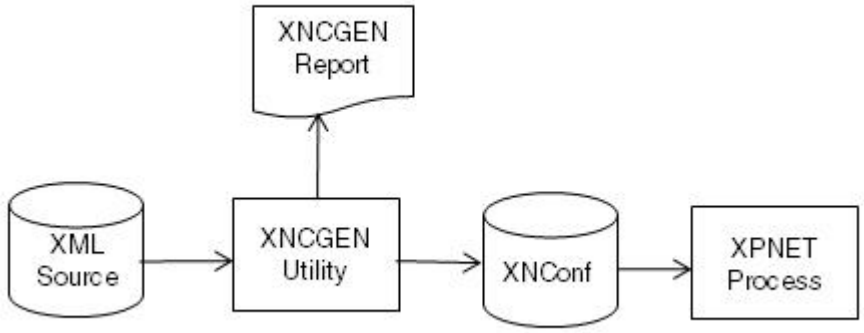


For additional information, refer to the *NET24-XPNET Command Profile Configuration Guide*.

Extended Network Configuration Generator (XNCGEN) utility

The XPNET process requires a compiled, extended segment file called the Extended Network Configuration (XNConf) to configure processes in the OSS space on an HP NonStop system and for starting any process which requires specific parameters to be sent to the application in the HP NonStop system startup message.

The Extended Network Configuration Generator (XNCGEN) utility is an offline batch program that reads an XML source file containing startup configuration information for one or more application processes and generates the XNConf. The XPNET process then uses the extended network configuration information contained in the XNConf created by the XNCGEN utility.



For additional information, refer to the *NET24-XPNET Extended Network Configuration Guide*.

License Verification utility (LIVERIFY)

A license verification utility is present on the XPNET subvolume to enable users to verify a license before loading it into a running system. This utility has the filename LIVERIFY. LIVERIFY can be run from the XPNET subvolume without any parameters and searches for the most recent license file in the same manner as XPNET searches. When it finds the most recent file, it opens it and reads it in the same manner as XPNET reads the license file. LIVERIFY provides detailed output including the following:

- The name of the license file
- The customer for whom the license was created
- The HP NonStop systems for which the license is valid
- A list of XPNET-related product IDs for which the customer is licensed
- The expiration date of the license (including the number of days left until expiration)
- A final indication of whether the license is valid on the system where LIVERIFY is being run

To verify licenses that are not located on the XPNET subvolume, volume to the subvolume where the license file is located and run the LIVERIFY utility specifying, as necessary, the path to the XPNET subvolume where the LIVERIFY utility is located. If a license file other than the most recent LIyymmdd license is to be verified, add an _ACI_LICENSE_FILE assign for that license before running the LIVERIFY utility. The GOLIVERI license verification obey file on the XPNET subvolume is provided to enable users to easily verify licenses with nonstandard names.

Network Audit Read utilities (NETADRD and NETADRT)

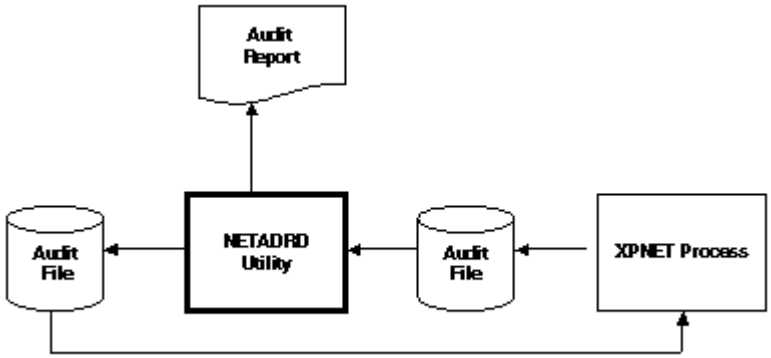
The NET24-XPNET product supports an audit facility that system analysts can use to trace message traffic within an XPNET system. If a system is experiencing problems, an audit can be performed so that message traffic can be analyzed.

This facility traces message traffic between designated stations and processes and the XPNET process. That is, it copies all messages between these designated stations and processes and the XPNET process to an audit file for subsequent analysis. This facility also provides for copying of individual messages marked for auditing by application processes to an audit file.

Large messages that XPNET manages as chunks are audited and displayed by NETADRD as separate chunks and not as a single large message.

You can use the NETADRD utility to create various types of output from an audit file or from a network overload management file. The output can be used for analyzing captured messages. Output can also be written to a disk file. Messages written to disk by the NETADRD utility can be reintroduced into the XPNET system.

The flow of the NETADRD utility is shown below.



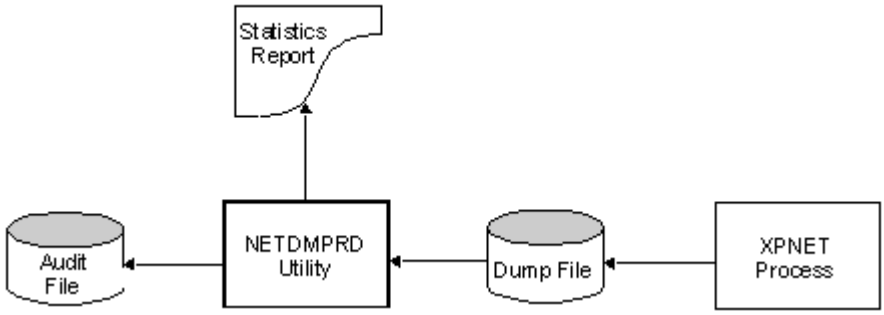
For more information on performing an audit and using the NETADRD utility, see the *NET24-XPNET Network Control Operations Guide*.

Network Dump Read utility (NETDMPRD)

The NETDMPRD utility is a recovery tool designed to extract any queued messages existing in an XPNET process saveabend file and write them to an audit file. You can then process the audit file using the NETADRD utility to create a report and, optionally, a disk file of the messages that can be reintroduced to the XPNET system.

You can specify two audit files as output. The first file is required and contains queued event messages, messages queued to the XPNET receive queue, and messages or chunks of large messages queued to processes, links, and stations, and to unknown destinations and unavailable services. The second audit file is optional and contains any messages that were queued to the audit queue at the time of the saveabend.

The NETDMPRD utility produces a report that contains a count of the number of messages written to the audit files. You can specify that the report be written to a spooler location or the terminal. The flow of the NETDMPRD utility is shown below.



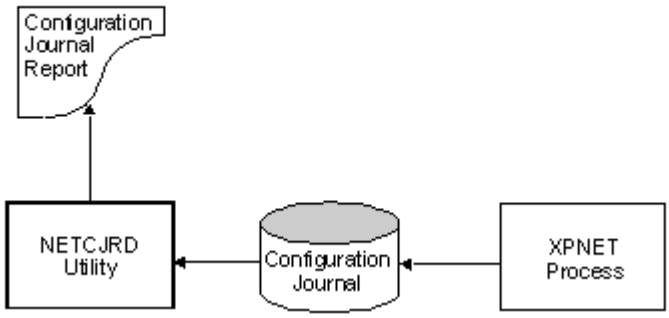
For more information on using the NETDMPRD utility, see the *NET24-XPNET Network Control Operations Guide*.

Network Configuration Journal Read utility (NETCJRD)

The XPNET process can optionally maintain a configuration journal for each XPNET node. This journal contains information about changes made to the configuration of objects on the XPNET node. You can use the NETCJRD utility to create a report from this configuration journal.

You can use this utility to create a report that contains changes made by a specified user, changes made between specified times, changes made by a specific command, and changes made to a specified object type.

The flow of the NETCJRD utility is shown below.



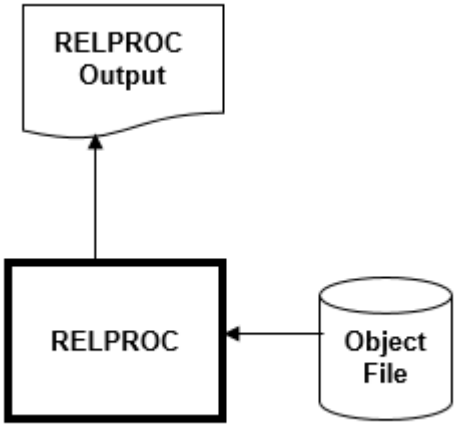
For more information on the configuration journal and using the NETCJRD utility, see the *NET24-XPNET Network Control Operations Guide*.

Release procedure macro (RELPROC)

Release procedures are placed into each object module to determine the release and fix version of XPNET software. By viewing the release procedure for a specific module, a customer can determine the code version and whether additional fixes are available for individual modules.

When a problem is called into the support center for XPNET, support personnel request the release procedure information for the affected modules. The RELPROC macro provides the mechanism to obtain release information.

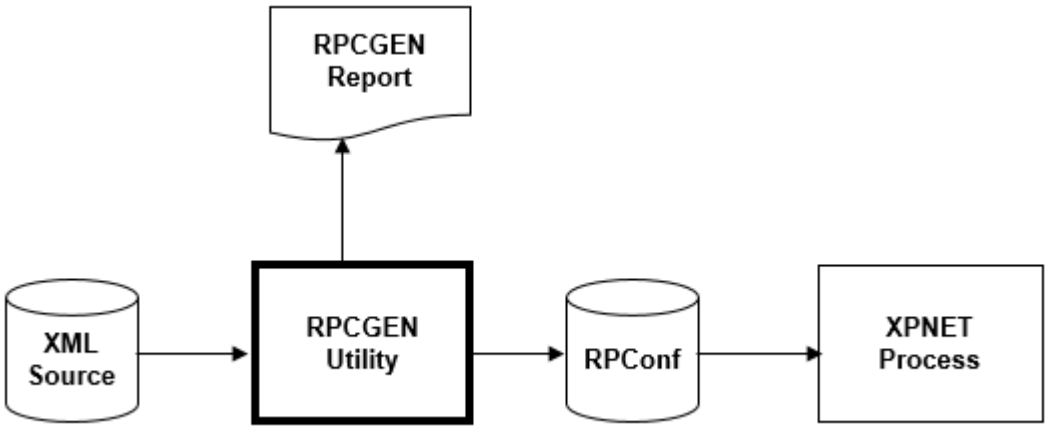
RELPROC uses BIND, NOFT, or eNOFT, depending on whether the object is TNS, TNS/R (native), or TNS/E (native), respectively, to obtain a list of the release procedures and display them at the terminal or in a specified file. RELPROC can provide abbreviated or detailed output for each release procedure identified. This information can be sent to the support center to provide assistance in solving the reported problem.



Routing Profile Configuration Generator (RPCGEN) utility

The XPNET process can use routing profile segments contained in a compiled, extended segment output file called the Routing Profile Configuration (RPConf) to configure complex message routing schemes for messages originating at stations and application processes within the network.

The Routing Profile Configuration Generator (RPCGEN) utility is an offline batch program that reads an XML source file with message routing configuration profiles, and generates the RPConf. The XPNET process then uses the routing profile information contained in the extended data segments created by the RPCGEN utility.



For additional information, refer to the *NET24-XPNET Routing Profile Configuration Guide*.

Section 5: ACI service-based architecture support

The XPNET system can be configured to support ACI service-based architecture (SBA) services, also known as business services integration (BSI). This configuration enables applications running on the HP NonStop platform to expose internal services, identify external services, and route messages between the two or to a dynamic service registry in the Universal Payments Platform (UPP). The implementation of the ACI service-based architecture takes advantage of XPNET service-based routing and broadcasting functionality.

Terminology

The following terms are used in ACI service-based architecture (SBA).

dynamic service registry (DSR)	A stand-alone process that keeps track of service providers and consumers that are written in Java and reside off of the HP NonStop platform. The HP NonStop system supports only the Service Locator process, which keeps track of consumers and providers written in C++. When multiple platforms are in use, the Service Locator process supports the capability to stay in synchronization with DSRs running on other platforms.
local service registry (LSR)	A SIS library that runs in the consumer and provider processes under the XPNET process. The LSR enables consumers to find providers of services (using the Service Locator process), helps providers register their services with the Service Locator process, provides a way for providers to find alternate consumers when a response to a consumer fails, and enables the provider services to be unregistered when a process is stopped.
service consumer	A process that uses (or consumes) services from another provider to accomplish a task.
Service Locator process (SVLOC)	A stand-alone process running under XPNET that keeps track of all service proiders in the system and enables service consumers to locate the providers they need to use.
service provider	A process that provides services necessary for message processing.
station pool	A group of stations that are used by service consumers or providers. There should be one station for each external service, one for each callback service, and one for each local service that is accessed from outside of the XPNET system.
Universal Payments Platform (UPP)	A general-purpose transaction processing platform that can be used for applications in different domains such as retail payments, wholesale payments, and merchant gateways.

Overview

The XPNET system's role in a service-based architecture environment is to pass messages among the service consumers, Service Locator process, and service providers. The consumers, locator, and providers reside on the HP NonStop platform. In addition, the XPNET process can route to service providers on another platform using the Universal Payments Platform (UPP).

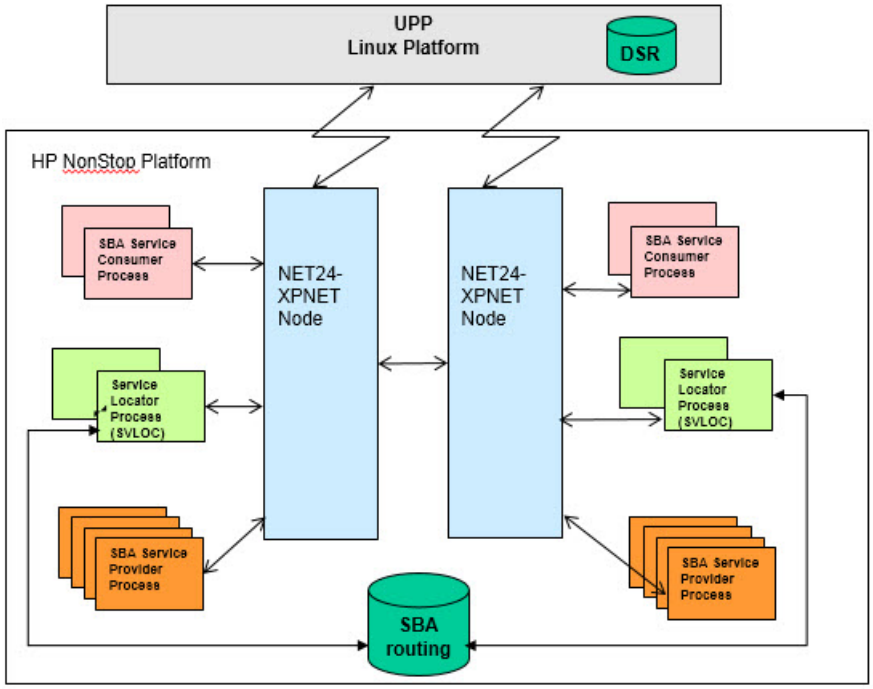
The XPNET process provides the following functions in a service-based architecture:

- Handling communication between the Service Locator process and DSR
- Handling service-consumer requests for locating services
- Registering services for service providers
- Routing status and routing information from the Service Locator process to service consumers

- Connecting to external service providers
- Connecting external service consumers to service providers on the HP NonStop system
- Notifying the Service Locator process of service provider availability changes
- Providing a mechanism for the Service Locator process to determine the status of service availability
- Providing the ability for the Service Locator process to determine if the system is HP NonStop only is also running on another platform with UPP
- Providing a facility to dynamically establish outbound and call back communication connections between applications running user XPNET on the HP NonStop and service providers and consumers running under UPP and off platform
- Providing the ability for SBA consumers running on the NonStop to send synchronous requests to service providers running on the NonStop and also under UPP and off platform
- Controlling security for which BSI services a consumer application can route messages to and which BSI services a provider application can register

The above functions are accomplished through the use of XPNET “well known” services that are shared across the Service Locator process and SBA applications.

The following diagram shows the XPNET process relationships with service consumers, Service Locator processes, service providers, and the DSR.



Well-known services

Three services facilitate locating services and communicating with one another. They are:

- SBALOCATE - used to determine which XPNET service to route messages to
- SBANOTIFY - used to notify the Service Locator processes of the available SBA services and the XPNET services associated with each of the services
- SBASTATUS - used to notify the SBA applications when an SBA service routing change has taken place

Service locators

The Service Locator process (SVLOC) is a stand-alone process on the HP NonStop platform, running under XPNET. This process is required on this platform when using SBA, regardless of whether UPP is installed. The

SVLOC enables consumers to find providers in the network on both the HP NonStop and other platforms (using UPP). If the providers are off the platform, SVLOC sends messages through XPNET to the UPP dynamic service registry (DSR) to find and make connections for off platform providers. SVLOC also assists with registering local providers with DSR so that UPP consumers can use services running on the HP NonStop platform under XPNET.

Service consumers

Service consumers are XPNET satellite process with components that consume one or more services. Each service consumer is known to the XPNET process by a consumer ID.

Consumers can register asynchronous callback functions for multiple consumer IDs with the Service Locator process. More than one XPNET satellite process can have a component that uses the same consumer ID.

Service consumers can also run off-platform and consume one or more SBA services available on the HP NonStop.

Service providers

Service providers are XPNET satellite processes with components that provide one or more services. Each variation of a service (such as by versions or attributes) is known to the XPNET process by a provider ID. Each service provider can register multiple provider IDs. More than one XPNET satellite process can register the same provider ID.

Service providers can also run off-platform and can be accessed through communication channels established by the LSR or SVLOC after being located through accessing the DSR.

What's inside

Inside the service consumer processes and the service provider processes is the local services registry (LSR). The LSR is a SIS library that runs in consumer and provider processes under XPNET. The LSR enables consumers to find providers of services (using SVLOC), helps providers register their services (with SVLOC), provides a way for providers to find alternate consumers when a response to a consumer fails, and enables the provider services to be unregistered when a process is stopped.

The SVLOC and the LSR are used to set up the dynamic connections using an SBACONNECT (for TCP clients) or SBACCEPT (for TCP servers) message. To do this, a pool of stations (or station pool) must be configured. To be a part of the pool, stations must have settings for the following attributes:

- SBAENABLE
- NOTIFYSERVICE
- UPPHDRVSN

Refer to the *NET24-XPNET Network Command Reference Manual* for more information about these attributes.

Processing

As required, the Service Locator process dynamically allocates connections out of the station pool using an internal API to XPNET (SBACONNECT and SBACCEPT).

Synchronous versus asynchronous processing

Service consumers and providers can run synchronously or asynchronously. Synchronous consumers and providers wait for a response. Asynchronous providers and consumers complete other work while they are waiting for a response. The message size for synchronous channels must be less than or equal to 32 KB. Asynchronous channels using TCP/IP can have message sizes up to 10 MB, although ACI recommends that the message size be limited to 1 MB or smaller because of performance issues.

In an asynchronous message, the ID length and message portions of the UPP header are set to 0. In a synchronous outbound (request) message, the message ID portion of the header is set to a value greater than 0, followed by the correlation data, and a correlation ID of 0. In an inbound (response) message, the correlation ID has a length that is set to a value greater than 0, followed by the returned message data and a message ID length of 0.

XPNET supports inbound and outbound synchronous requests for local service consumers accessing off platform service providers.

If the object file has PROGID enabled, the owner (PAID) of the running process will be the Guardian owner of the object file. If PROGID is not enabled, the PAID will be the user that started the XPNET process (ancestor) that is managing this process. This PAID is used to control the security (SBAS and SBAP) of the BSI consumer and provider applications. If different levels of security are required, different PROGID objects can be used.

Implementation tasks

To set up your XPNET system for service based architecture, do the following:

1. Set up security by setting up security profiles and the security user file.
2. Define the stations in the station pool.
3. Configure TCP/IP connection (station) between XPNET and the UPP DSR, if using.
4. Start the TCP/IP connection between XPNET and the UPP DSR.
5. Add a SVLOC process under XPNET (one per XPNET) node.
6. Configure the Service Locator process. Refer to integration documentation for information about setting up the Service Locator process.

Setting up XPNET security

The services that the XPNET node can access and provide depend on the security settings of the Guardian ID of the owner of the application attempting to consume or provide services (that is, the person who starts the XPNET system). The security is based on profiles containing services that can be accessed (or not accessed) or denied. If the object file is PROGID enabled, the owner (PAID) of the running process is the user that started the XPNET process (ancestor) that is managing this process.

For each user, you can configure one of the following access types to each of the consumer service profiles listed for that user:

- E = Everything. The user has access to all services. No service profile records are required.
- N = None. The user has no access to any of the services. No service profile records are required.
- D = Deny. The user is denied access to the services listed in the specified profiles.
- A = Allow. The user is allowed access to the services listed in the specified profiles.

Each service profile can contain up to 250 services. You can assign up to 20 DEST (or consumer) and 20 Registered (or provider) service profiles per user, giving control of 10,000 services to a single ID.

In addition, you can configure for each user one of the following access types to each of the service provider profiles listed for that user:

- E = Everything. The user has access to all services. No service profile records are required.
- N = None. The user has no access to any of the services. No service profile records are required.
- D = Deny. The user is denied access to the services listed in the specified profiles.
- A = Allow. The user is allowed access to the services listed in the specified profiles.

You can assign up to 20 DEST (or consumer) and 20 Registered (or provider) service provider profiles per user.

When you make changes to the security files, you must stop and start the application for the changes to take effect.

Set up service-based security profiles

Service-based security profiles are configured in the stand-alone SBA Profile Security File (XPSBAP). There is also a pathway requester/server version of this available under the NET24 Pathway subsystem. SBA profile records specify which services a provider can register (or not register), and indicate whether each service can be accessed (or not accessed) by the consumer. After a security profile is created and stored in the SBAP, it can be associated on the stand-alone SBA User Security File (XPSBAS) screen with the security record for a user.

Before you begin:

It is helpful to have an understanding of the services you would like to group into the profiles. Remember that these profiles can be used for many different users and scenarios. For example, if you want a user ID to be able to use only a select group of services, you can put them together in a profile. That profile can also be used when you want a user ID to be able to access everything accept those services.

Note: Changes made to an SBA profile affect all users with that security profile named in their SBAS record. For example, if there are five users with the SBAP profile DEBITADD specified in their SBAS records and you increase the number of services accessible to the DEBITADD profile, all five users have increased access. To increase the access for only one of the five users, you must assign a different security profile to that one user. As this is controlled by the PAID of the process (either the XPNET node) or the object (PROGID), this may require multiple PROGID object files owned by different NSK userid's (user num, user group) to work correctly.

1. Type XPSBAP at the TACL prompt to go to the XPSBAP window. There is also a Pathway requester/server version of this available under the NET24 Pathway subsystem.
2. Type the name of the profile in the PROFILE field.
3. Type the names of all of the services in the SBA SECURITY LIST fields. You can add 64 services on a page. If you need to go to subsequent pages, press the F9 key to go to the next page.
4. Press the F3 key to save the profile.
5. Repeat for each profile.

What to do next:

Assign profiles to a user.

Set up SBA user security files

The user security file contains the profiles that define which services a user can access.

Before you begin:

Become familiar with the service-based profiles that are available. Note that you can use a profile to allow access or deny access.

1. Enter XPSBAS at the TACL prompt to navigate to the XPSBAS window. There is also a Pathway requester/server version of this available under the NET24 Pathway subsystem.
2. Enter the SBAS file name and the group and member number for the user. This represents the Guardian ID of the person who logs on and starts the XPNET system or the owner of the object file (if it is PROGID).
3. Enter the access type. Valid values are as follows:
   - A = Allow. You allow access for this user only to the services in the listed profiles.
   - D = Deny. You allow access for this user to all services except those in the listed profiles.
   - E = Everything> You allow access for this user to all services. If you enter this code, there is no need to enter specific profiles
   - N = None. you deny access for this user to all services. If you enter this code, there is no need to enter specific profiles

4. If you entered E or N as the access type, skip to the next step. If you entered A or D, enter the profiles that you allow or deny access to this user. You can associate up to 20 DEST (or consumer) service profiles and up to 20 Registered (or provider) service profiles with a user.
5. Press the F3 key to add the record.

Define the stations in the station pool

Each service must have at least three stations defined: one for outbound messages, and two for inbound messages.

Before you begin:

Examine the stations that are already in your system to determine if the service-based architecture routing uses these stations or if you need to create new stations. This task assumes that you have already added all services. You should have the following information ready for each station:

- For the SBAENABLE attribute, whether the station is dynamic (part of the pool with configuration data that the XPNET process can alter programmatically) or static (part of the pool and pre-configured data that the XPNET process cannot alter programmatically).
- For the NOTIFYSERVICE attribute, the 16-byte symbolic name of the SBANOTIFY service provided by the SVLOC service.
- For the UPPHDRVSN attribute, whether to use the initial support version of the UPP header (2) or not expect the header at all (0). You must set this value to 2 if the XPNET process uses the station for routing to a DSR or any off-platform consumer or provider.

Service-based architecture relies on a pool of stations that the XPNET process can use in routing messages between off-platform service consumers and service providers. The stations within the pool can be dynamic or static. The XPNET process can programmatically change dynamic stations to route messages to a service consumer or provider as needed. You configure static stations to route messages to a specific service. When the XPNET process searches for a station and finds a static station that does not match the service destination, it continues looking for a match or a dynamic station.

1. Use the ALTER STATION command (or the SET STATION COMMAND if you are in an obey file) for the station as described in the *NET24-XPNET Network Control Command Reference*. Specify the SBAENABLE attribute. Set the value to DYNAMIC or STATIC. Enter the command.
2. Enter the ALTER STATION command again, but this time use the NOTIFYSERVICE attribute with the symbolic name of the SBANOTIFY service.
3. Enter the command again, but this time use the UPPHDRVSN attribute with the value 2.
4. Set up the dynamic stations using the INITDSTS, INTVALDSTS, INITDSTS, INTVALDSTS, and TEMPLATE attributes. Refer to the *NET24-XPNET Internal TCP-IP Implementation Guide* for more information on dynamic stations.

What to do next:

Repeat the process for each station in the station pool.

Configure the TCP/IP connection between XPNET and the UPP dynamic service registry (DSR)

To communicate with the UPP DSR, a TCP/IP connection between the XPNET process and the UPP must be configured.

Before you begin. Gather the following information for the UPP DSR:

- IP address (or host name)
- Port number

Configure the connection as described in the *XPNET Internal TCP/IP Protocol Implementation Guide*.

Start the TCP/IP connection between XPNET and the UPP DSR

After the TCP/IP connections to the UPP DSR is established, it must be started. Refer to the *NET24-XPNET Network Control Command Reference* and the *XPNET Internal TCP/IP Protocol Implementation Guide* for instructions for starting the connection.

Add a SVLOC process under XPNET

The server locator (SVLOC) process must be added to the XPNET system. Refer to the *NET24-XPNET Network Control Operations Guide* for information about defining and adding objects to an XPNET system.

Appendix A: **Third-party software license agreements**

This ACI Software Product is licensed to you or your company or organization (“You”) under the terms of the written license agreement in place between You and ACI (the “License Agreement”).

In addition to the License Agreement, the third party license terms displayed below govern ACI’s distribution to You and Your use of the individual third party software components included with or in the ACI Software Product, but do NOT supersede or limit the terms and conditions of the License Agreement for use of the ACI Software Product. The following third party license terms shall be deemed to apply exclusively to the applicable third party software associated with such third party license and NOT to the ACI Software Product, or any portion thereof, with which such third party software is used or distributed. Any of the following third party license terms that differ from or conflict with the License Agreement shall not affect the License Agreement or your license thereunder in the ACI Software Product. As between You and ACI, the ACI Software Product shall not be deemed to be a derivative work or modification of, or a contribution to, any of the third party software used or distributed with such ACI Software Product.

The following third party license terms are excerpted from the license file for the applicable third party software component; see the full license terms included with the Software Product for specific language governing permissions and limitations under the third party license for each such third party software component.

The following table shows the third-party software for the NET24-XPNET product:

Software	Version	License version
ANTLR	2.7.1	*ANTLR Software Rights Notice*
Apache log4j	1.0.4	*Apache License 1.1*
Apache Xerces2 J	1.4.0	*Apache License 2.0*
exolabcore	0.3.5	*Exoffice License*
Java Message Service	1.0.2b	*Sun Java Message Service 1.0.2 License*
Java Transaction API (JTA)	1.0.1	*JTA 101 Sun License*
JDBM	0.081	*BSD 3-clause "New" or "Revised" License*
OpenJMS	openjms-0.7.2b14	*Exoffice License*
OpenJMS	openjms-0.7.3	*Exoffice License*
OpenJMS	openjms-0.7.5-rc1	*Exoffice License*
ots-jts	1.0	*Exoffice License*
saxpath	1.0	*Saxpath License*
Tyrex	1.0.1	*Tyrex License (Vovida License +)*

ANTLR Software Rights Notice

(ANTLR 2.7.1)

ANTLR License

==========

SOFTWARE RIGHTS

ANTLR 1989-2004 Developed by Terence Parr Partially supported by University of San Francisco & jGuru.com

We reserve no legal rights to the ANTLR--it is fully in the public domain. An individual or company may do whatever they wish with source code distributed with ANTLR or the code generated by ANTLR, including the incorporation of ANTLR, or its output, into commerical software.

We encourage users to develop software with ANTLR. However, we do ask that credit is given to us for developing ANTLR. By "credit", we mean that if you use ANTLR or incorporate any source code into one of your programs (commercial product, research project, or otherwise) that you acknowledge this fact somewhere in the documentation, research report, etc... If you like ANTLR and have developed a nice tool with the output, please mention that you developed it using ANTLR. In addition, we ask that the headers remain intact in our source code. As long as these guidelines are kept, we expect to continue enhancing this system and expect to make other tools available as they are completed.

The primary ANTLR guy:

Terence Parr

parrt@cs.usfca.edu

parrt@antlr.org

Apache License 1.1

(Apache log4j 1.0.4)

Apache Software License

==========

Version 1.1

Copyright (c) 2000 The Apache Software Foundation. All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
3. The end-user documentation included with the redistribution, if any, must include the following acknowledgment:

   "This product includes software developed by the Apache Software Foundation (*http://www.apache.org/*)."

   Alternately, this acknowledgment may appear in the software itself, if and wherever such third-party acknowledgments normally appear.
4. The names "Apache" and "Apache Software Foundation" must not be used to endorse or promote products derived from this software without prior written permission. For written permission, please contact *apache@apache.org*.
5. Products derived from this software may not be called "Apache", nor may "Apache" appear in their name, without prior written permission of the Apache Software Foundation.

THIS SOFTWARE IS PROVIDED "AS IS" AND ANY EXPRESSED OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE APACHE SOFTWARE FOUNDATION OR ITS CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER

CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

This software consists of voluntary contributions made by many individuals on behalf of the Apache Software Foundation. For more information on the Apache Software Foundation, please see *<http://www.apache.org/>*.

Portions of this software are based upon public domain software originally written at the National Center for Supercomputing Applications, University of Illinois, Urbana-Champaign.

Apache License 2.0

(Apache Xerces2 J 1.4.0)

Apache License

Version 2.0, January 2004

==========

http://www.apache.org/licenses/

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

   "License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

   "Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License. "Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50%) or more of the outstanding shares, or (iii) beneficial ownership of such entity.

   "You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

   "Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

   "Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

   "Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

   "Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

   "Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to

communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.
3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.
4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

   a. You must give any other recipients of the Work or Derivative Works a copy of this License; and
   b. You must cause any modified files to carry prominent notices stating that You changed the files; and
   c. You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and
   d. If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.

      You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.
5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions..
6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.
7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or

conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.
9. Accepting Warranty or Additional Liability. While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

APPENDIX: How to apply the Apache License to your work

To apply the Apache License to your work, attach the following boilerplate notice, with the fields enclosed by brackets "[]" replaced with your own identifying information. (Don't include the brackets!) The text should be enclosed in the appropriate comment syntax for the file format. We also recommend that a file or class name and description of purpose be included on the same "printed page" as the copyright notice for easier identification within third-party archives.

Copyright [yyyy] [name of copyright owner] Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at *http://www.apache.org/licenses/LICENSE-2.0* Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

BSD 3-clause "New" or "Revised" License

(JDBM 0.081)

Copyright (c) <YEAR>, <OWNER>

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

* Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

* Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

* Neither the name of the <ORGANIZATION> nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO,

PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Exoffice License

(exolabcore 0.3.5, OpenJMS openjms-0.7.2b14, OpenJMS openjms-0.7.3, OpenJMS openjms-0.7.5-rc1, ots-jts 1.0)

Exoffice License

==========

Copyright 1999 (C) Exoffice Technologies Inc. All Rights Reserved.

Redistribution and use of this software and associated documentation ("Software"), with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain copyright statements and notices. Redistributions must also contain a copy of this document.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
3. The name "Exolab" must not be used to endorse or promote products derived from this Software without prior written permission of Exoffice Technologies. For written permission, please contact *info@exolab.org*.
4. Products derived from this Software may not be called "Exolab" nor may "Exolab" appear in their names without prior written permission of Exoffice Technologies. Exolab is a registered trademark of Exoffice Technologies..
5. Due credit should be given to the Exolab Project (*http://www.exolab.org/*).

THIS SOFTWARE IS PROVIDED BY EXOFFICE TECHNOLOGIES AND CONTRIBUTORS "AS IS" AND ANY EXPRESSED OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL EXOFFICE TECHNOLOGIES OR ITS CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

JTA 101 Sun License

(Java Transaction API (JTA) 1.0.1)

jta 101 Sun License

==========

License Agreement

SUN MICROSYSTEMS, INC. ("SUN") IS WILLING TO LICENSE ITS Java Transaction API SOFTWARE ("SOFTWARE") TO YOU ("CUSTOMER") ONLY UPON THE CONDITION THAT YOU ACCEPT ALL OF THE TERMS CONTAINED IN THIS LICENSE AGREEMENT ("AGREEMENT"). READ THE TERMS AND CONDITIONS OF THE AGREEMENT CAREFULLY BEFORE SELECTING THE "ACCEPT" BUTTON AT THE BOTTOM OF THIS PAGE. BY SELECTING THE "ACCEPT" BUTTON YOU AGREE TO THE TERMS AND CONDITIONS OF THE AGREEMENT. IF YOU ARE NOT WILLING TO BE BOUND BY ITS TERMS, SELECT

THE "DO NOT ACCEPT" BUTTON AT THE BOTTOM OF THIS PAGE AND THE INSTALLATION PROCESS WILL NOT CONTINUE.

1. License to Distribute. Customer is granted a royalty-free, non-transferable right to reproduce and use the Software for the purpose of developing applications which run in conjunction with the Software. Customer may not modify the Software (including any APIs exposed by the Software) in any way.
2. Restrictions. Software is confidential copyrighted information of Sun and title to all copies is retained by Sun and/or its licensors. Except to the extent enforcement of this provision is prohibited by applicable law, if at all, Customer shall not decompile, disassemble, decrypt, extract, or otherwise reverse engineer Software. Software is not designed or intended for use in on-line control of aircraft, air traffic, aircraft navigation or aircraft communications; or in the design, construction, operation or maintenance of any nuclear facility. Customer warrants that it will not use or redistribute the Software for such purposes.
3. Trademarks and Logos. This Agreement does not authorize Customer to use any Sun name, trademark or logo. Customer acknowledges that Sun owns the Java trademark and all Java-related trademarks, logos and icons including the Coffee Cup and Duke ("Java Marks") and agrees to:

   i. comply with the Java Trademark Guidelines at

   http://java.sun.com/trademarks.html;

   ii. not do anything harmful to or inconsistent with Sun's rights in the Java Marks; and

   iii. assist Sun in protecting those rights, including assigning to Sun any rights acquired by Customer in any Java Mark.
4. Disclaimer of Warranty. Software is provided "AS IS," without a warranty of any kind. ALL EXPRESS OR IMPLIED REPRESENTATIONS AND WARRANTIES, INCLUDING ANY IMPLIED WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT, ARE HEREBY EXCLUDED.
5. Limitation of Liability. IN NO EVENT WILL SUN OR ITS LICENSORS BE LIABLE FOR ANY LOST REVENUE, PROFIT OR DATA, OR FOR SPECIAL, INDIRECT, CONSEQUENTIAL, INCIDENTAL OR PUNITIVE DAMAGES HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY ARISING OUT OF THE DOWNLOADING OF, USE OF, OR INABILITY TO USE, SOFTWARE, EVEN IF SUN HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.
6. Termination. Customer may terminate this Agreement at any time by destroying all copies of Software. This Agreement will terminate immediately without notice from Sun if Customer fails to comply with any provision of this Agreement. Upon such termination, Customer must destroy all copies of Software. Sections 4 and 5 above shall survive termination of this Agreement.
7. Export Regulations. Software, including technical data, is subject to U.S. export control laws, including the U.S. Export Administration Act and its associated regulations, and may be subject to export or import regulations in other countries. Customer agrees to comply strictly with all such regulations and acknowledges that it has the responsibility to obtain licenses to export, re-export, or import Software. Software may not be downloaded, or otherwise exported or re-exported

   i. into, or to a national or resident of, Cuba, Iraq, Iran, North Korea, Libya, Sudan, Syria or any country to which the U.S. has embargoed goods; or

   ii. to anyone on the U.S. Treasury Department's list of Specially Designated Nations or the U.S. Commerce Department's Table of Denial Orders.
8. Restricted Rights. Use, duplication or disclosure by the United States government is subject to the restrictions as set forth in the Rights in Technical Data and Computer Software Clauses in DFARS 252.227-7013(c) (1) (ii) and FAR 52.227-19(c) (2) as applicable.
9. Governing Law. Any action related to this Agreement will be governed by California law and controlling U.S. federal law. No choice of law rules of any jurisdiction will apply.
10. Severability. If any of the above provisions are held to be in violation of applicable law, void, or unenforceable in any jurisdiction, then such provisions are herewith waived or amended to the extent necessary for the Agreement to be otherwise enforceable in such jurisdiction. However, if in Sun's opinion deletion or

amendment of any provisions of the Agreement by operation of this paragraph unreasonably compromises the rights or increase the liabilities of Sun or its licensors, Sun reserves the right to terminate the Agreement.

Saxpath License

(saxpath 1.0)

Saxpath License

==========

Copyright (C) 2000-2002 werken digital.

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions, and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions, and the disclaimer that follows these conditions in the documentation and/or other materials provided with the distribution.
3. The name "SAXPath" must not be used to endorse or promote products derived from this software without prior written permission. For written permission, please contact *license@saxpath.org*.
4. Products derived from this software may not be called "SAXPath", nor may "SAXPath" appear in their name, without prior written permission from the SAXPath Project Management (*pm@saxpath.org*).

In addition, we request (but do not require) that you include in the end-user documentation provided with the redistribution and/or in the software itself an acknowledgement equivalent to the following:

"This product includes software developed by the SAXPath Project

(*http://www.saxpath.org/*)."

Alternatively, the acknowledgment may be graphical using the logos available at *http://www.saxpath.org/*

THIS SOFTWARE IS PROVIDED ``AS IS'' AND ANY EXPRESSED OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE SAXPath AUTHORS OR THE PROJECT CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Sun Java Message Service 1.0.2 License

(Java Message Service 1.0.2b)

License Agreement for Java(TM) Message Service (JMS) 1.0.2b

==========

Sun Microsystems, Inc.

Binary Code License Agreement

READ THE TERMS OF THIS AGREEMENT AND ANY PROVIDED SUPPLEMENTAL LICENSE TERMS (COLLECTIVELY "AGREEMENT") CAREFULLY BEFORE OPENING THE SOFTWARE MEDIA PACKAGE. BY OPENING THE SOFTWARE MEDIA PACKAGE, YOU AGREE TO THE TERMS OF THIS AGREEMENT. IF YOU ARE ACCESSING THE SOFTWARE ELECTRONICALLY, INDICATE YOUR ACCEPTANCE OF THESE TERMS BY SELECTING THE "ACCEPT" BUTTON AT THE END OF THIS AGREEMENT. IF YOU DO NOT AGREE TO ALL THESE TERMS, PROMPTLY RETURN THE UNUSED SOFTWARE TO YOUR PLACE OF PURCHASE FOR A REFUND OR, IF THE SOFTWARE IS ACCESSED ELECTRONICALLY, SELECT THE "DECLINE" BUTTON AT THE END OF THIS AGREEMENT.

1. LICENSE TO USE. Sun grants you a non-exclusive and non-transferable license for the internal use only of the accompanying software and documentation and any error corrections provided by Sun (collectively "Software"), by the number of users and the class of computer hardware for which the corresponding fee has been paid.
2. RESTRICTIONS. Software is confidential and copyrighted. Title to Software and all associated intellectual property rights is retained by Sun and/or its licensors. Except as specifically authorized in any Supplemental License Terms, you may not make copies of Software, other than a single copy of Software for archival purposes. Unless enforcement is prohibited by applicable law, you may not modify, decompile, or reverse engineer Software. You acknowledge that Software is not designed, licensed or intended for use in the design, construction, operation or maintenance of any nuclear facility. Sun disclaims any express or implied warranty of fitness for such uses. No right, title or interest in or to any trademark, service mark, logo or trade name of Sun or its licensors is granted under this Agreement.
3. LIMITED WARRANTY. Sun warrants to you that for a period of ninety (90) days from the date of purchase, as evidenced by a copy of the receipt, the media on which Software is furnished (if any) will be free of defects in materials and workmanship under normal use. Except for the foregoing, Software is provided "AS IS". Your exclusive remedy and Sun's entire liability under this limited warranty will be at Sun's option to replace Software media or refund the fee paid for Software.
4. DISCLAIMER OF WARRANTY. UNLESS SPECIFIED IN THIS AGREEMENT, ALL EXPRESS OR IMPLIED CONDITIONS, REPRESENTATIONS AND WARRANTIES, INCLUDING ANY IMPLIED WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT ARE DISCLAIMED, EXCEPT TO THE EXTENT THAT THESE DISCLAIMERS ARE HELD TO BE LEGALLY INVALID.
5. LIMITATION OF LIABILITY. TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL SUN OR ITS LICENSORS BE LIABLE FOR ANY LOST REVENUE, PROFIT OR DATA, OR FOR SPECIAL, INDIRECT, CONSEQUENTIAL, INCIDENTAL OR PUNITIVE DAMAGES, HOWEVER CAUSED REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF OR RELATED TO THE USE OF OR INABILITY TO USE SOFTWARE, EVEN IF SUN HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. In no event will Sun's liability to you, whether in contract, tort (including negligence), or otherwise, exceed the amount paid by you for Software under this Agreement. The foregoing limitations will apply even if the above stated warranty fails of its essential purpose.
6. Termination. This Agreement is effective until terminated. You may terminate this Agreement at any time by destroying all copies of Software. This Agreement will terminate immediately without notice from Sun if you fail to comply with any provision of this Agreement. Upon Termination, you must destroy all copies of Software.
7. Export Regulations. All Software and technical data delivered under this Agreement are subject to US export control laws and may be subject to export or import regulations in other countries. You agree to comply strictly with all such laws and regulations and acknowledge that you have the responsibility to obtain such licenses to export, re-export, or import as may be required after delivery to you.
8. U.S. Government Restricted Rights. If Software is being acquired by or on behalf of the U.S. Government or by a U.S. Government prime contractor or subcontractor (at any tier), then the Government's rights in Software and accompanying documentation will be only as set forth in this Agreement; this is in accordance with 48 CFR 227.7201 through 227.7202-4 (for Department of Defense (DOD) acquisitions) and with 48 CFR 2.101 and 12.212 (for non-DOD acquisitions).
9. Governing Law. Any action related to this Agreement will be governed by California law and controlling U.S. federal law. No choice of law rules of any jurisdiction will apply.

10. Severability. If any provision of this Agreement is held to be unenforceable, this Agreement will remain in effect with the provision omitted, unless omission would frustrate the intent of the parties, in which case this Agreement will immediately terminate.
11. Integration. This Agreement is the entire agreement between you and Sun relating to its subject matter. It supersedes all prior or contemporaneous oral or written communications, proposals, representations and warranties and prevails over any conflicting or additional terms of any quote, order, acknowledgment, or other communication between the parties relating to its subject matter during the term of this Agreement. No modification of this Agreement will be binding, unless in writing and signed by an authorized representative of each party.

JAVA(TM) INTERFACE CLASSES

JAVA MESSAGE SERVICE (JMS), VERSION 1.0.2

SUPPLEMENTAL LICENSE TERMS

These supplemental license terms ("Supplemental Terms") add to or modify the terms of the Binary Code License Agreement (collectively, the "Agreement"). Capitalized terms not defined in these Supplemental Terms shall have the same meanings ascribed to them in the Agreement. These Supplemental Terms shall supersede any inconsistent or conflicting terms in the Agreement, or in any license contained within the Software.

1. Software Internal Use and Development License Grant. Subject to the terms and conditions of this Agreement, including, but not limited to Section 3 (Java(TM) Technology Restrictions) of these Supplemental Terms, Sun grants you a non-exclusive, non-transferable, limited license to reproduce internally and use internally the binary form of the Software, complete and unmodified, for the sole purpose of designing, developing and testing your Java applets and applications ("Programs").
2. License to Distribute Software. In addition to the license granted in Section 1 (Software Internal Use and Development License Grant) of these Supplemental Terms, subject to the terms and conditions of this Agreement, including but not limited to Section 3 (Java Technology Restrictions), Sun grants you a non-exclusive, non-transferable, limited license to reproduce and distribute the Software in binary form only, provided that you

   i. distribute the Software complete and unmodified and only bundled as part of your Programs,

   ii. do not distribute additional software intended to replace any component(s) of the Software,

   iii. do not remove or alter any proprietary legends or notices contained in the Software,

   iv. only distribute the Software subject to a license agreement that protects Sun's interests consistent with the terms contained in this Agreement, and

   v. agree to defend and indemnify Sun and its licensors from and against any damages, costs, liabilities, settlement amounts and/or expenses (including attorneys' fees) incurred in connection with any claim, lawsuit or action by any third party that arises or results from the use or distribution of any and all Programs and/or Software.
3. Java Technology Restrictions. You may not modify the Java Platform Interface ("JPI", identified as classes contained within the "java" package or any subpackages of the "java" package), by creating additional classes within the JPI or otherwise causing the addition to or modification of the classes in the JPI. In the event that you create an additional class and associated API(s) which

   i. extends the functionality of the Java Platform, and

   ii. is exposed to third party software developers for the purpose of developing additional software which invokes such additional API, you must promptly publish broadly an accurate specification for such API for free use by all developers. You may not create, or authorize your licensees to create additional classes, interfaces, or subpackages that are in any way identified as "java", "javax", "sun" or similar convention as specified by Sun in any naming convention designation.
4. Trademarks and Logos. You acknowledge and agree as between you and Sun that Sun owns the SUN, SOLARIS, JAVA, JINI, FORTE, STAROFFICE, STARPORTAL and iPLANET trademarks and all SUN,

SOLARIS, JAVA, JINI, FORTE, STAROFFICE, STARPORTAL and iPLANET-related trademarks, service marks, logos and other brand designations ("Sun Marks"), and you agree to comply with the Sun Trademark and Logo Usage Requirements currently located at *http://www.sun.com/policies/trademarks*. Any use you make of the Sun Marks inures to Sun's benefit.

5. Source Code. Software may contain source code that is provided solely for reference purposes pursuant to the terms of this Agreement. Source code may not be redistributed unless expressly provided for in this Agreement.
6. Termination for Infringement. Either party may terminate this Agreement immediately should any Software become, or in either party's opinion be likely to become, the subject of a claim of infringement of any intellectual property right.

For inquiries please contact: Sun Microsystems, Inc. 901 San Antonio Road, Palo Alto, California 94303

(Form ID#011801)

Tyrex License (Vovida License +)

(Tyrex 1.0.1)

Tyrex License

==========

Original code is Copyright (c) 1999-2001, Intalio, Inc. All Rights Reserved.

Contributions by MetaBoss team are Copyright (c) 2003-2004, Softaris Pty. Ltd.

All Rights Reserved. Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
3. The name "Exolab" must not be used to endorse or promote products derived from this Software without prior written permission of Intalio. For written permission, please contact info@exolab.org..
4. Products derived from this Software may not be called "Exolab" nor may "Exolab" appear in their names without prior written permission of Intalio. Exolab is a registered trademark of Intalio.
5. Due credit should be given to the Exolab Project (*http://www.exolab.org/*).

THIS SOFTWARE IS PROVIDED BY INTALIO AND CONTRIBUTORS ``AS IS'' AND ANY EXPRESSED OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL INTALIO OR ITS CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.